AIN SHAMS UNIVERSITY

FACULTY OF ENGINEERING

Electronics Engineering and Electrical Communications

ASIC implementation of RISC-V based Microcontroller [For wireless charging applications]

A Thesis submitted in partial fulfillment of the requirements of the degree of

Bachelor's in electrical engineering

(Electronics Engineering and Electrical Communications)

by

Mohamed Tarek Mohamed 1900889

Faculty of Engineering, Ain-Shams University, year 2023-2024

Supervised By

Prof. Hani Fikry

Eng. Islam Samir (ICpedia)

Acknowledgement

This thesis of this BSc. Graduation project wouldn’t have been possible without the efforts of Professor Hani Fikry and the support of ICpedia PnR team led by Eng. Islam Samir. It is a pleasure to thank who made this possible

Contents

1.1 Introduction 9
1.1.1 PULP-Platform: 9
1.1.2 What about The PULPino Project: 9
1.1.3 Ibex (Zero-riscy) Core for PULPino Project: 10
1.1.4 Key Features of the Ibex (Zero-riscy) Core: 11
1.1.5 Applications of Pulpino with Ibex (Zero-riscy) core: 12
1.1.6 RI5CY Core for PULPino Project: 12
1.1.7Key Features of the RI5CY Core 13
1.1.8 Applications of RI5CY 13
1.1.9 The project vision: 14
2.1 Introduction to Synthesis and Design Compiler 15
2.1.1 Key Concepts in Synthesis 15
2.1.2 Typical Synthesis Workflow 16
2.1.3 Importance of Synthesis in Digital Design 17
2.1.4 Design Compiler: Features and Workflow 17
2.1.5 Design compiler different flows 17
2.1.6 Flat Flow 18
2.1.7 Hierarchical Flow 18
2.1.8 SAED 14nm technology 19
2.1.9 Synthesis Commands and Explanations: 20
2.1.9.1 Constrains commands and explanation: 20
2.1.9.2 Synthesis setup commands and explanation: 24
2.1.9.3 Elaboration commands and explanation: 25
2.1.9.4 Finalizing the synthesis 26
2.1.9.5 Compile the design 26
2.1.9.6 Detailed explanation of the compile_ultra options 27
2.1.9.7“compile_ultra” Command with “-no_autoungroup” Option 28

2.1.9.8 Write Out Files 28
3.1 IC Compiler II: 29
3.2 overview on PnR flow: 30
3.2.1 Floorplanning 30
3.2.2 Power Planning & Boundary Cell Insertion 30
3.2.3 Placement 30
3.2.4 CTS and Post-CTS Logical Optimization 31
3.2.5 Routing 31
3.2.6 PrimeTime 31
3.3 Floorplanning Overview: 31
3.3.1 Metal Layers Definition 32
3.3.2 IP Floorplanning 32
3.3.3 TAP Cells 32
3.3.4 Standard cell placement and Routing 32
3.3.5 Guidelines for a Good Floorplan 32
3.4 Floorplanning Commands: 32
4.1 Powerplanning 38
4.1.1 powerplanning overview: 38
4.1.2 power planning commands: 38
5.1.3 Placement optimization: 43
5.1.4 Tie-Cell Insertion 44
7.1 Routing 56
7.1.1 Routing overview 56
7.1.2 Routing main commands in PnR flow: 57
8.1 Chip Finishing 59
8.1.1 Chip finishing overview 59
8.1.2 Finishing main commands: 60
8.1.3 Reports and checks: 61

9.1 PRIME TIME 62
9.1.1 STA Overview 62
9.1.2 PrimeTime Overview 63
9.1.3 Key Features of PrimeTime 63
9.1.3.1 Verify timing 63
9.1.3.2 Is Exhaustive 63
9.1.3.3 Is Fast 63
9.1.4 Incremental Timing Analysis 63
9.1.5 ECO (Engineering Change Order) Support 63
9.1.6 Prime Time in the implementation flow 63
9.1.7 Prime time inputs and outputs 64
9.1.8 Timing Analysis Flow in Prime Time 65
9.1.9 Load design and check 65
9.1.10 Load Libraries and check 65
9.1.11Read parasitic 66
9.1.12 source constraints and check for corners 66
9.1.13 confirm constrints completeness 66
9.1.14 Update Timing 67
9.2 ECO(ENGINEERING CHANGE ORDER) 68
9.2.1 Overview of ECO in VLSI Design 68
9.2.2 Fixing Timing Violations 68
9.2.3 Addressing DRC Violations 68
9.2.4 Exporting ECO Changes 69
9.2.5 Applying ECO Changes in ICC2 69
9.2.6 Place ECO Cells: 69
9.2.7 Route ECO Changes: 70
9.2.8 Connect Power and Ground Nets: 70
9.2.9 Iterative ECO Verification and Signoff 70

10 Implemented PULPino SoC 70
10.1. Choosing RISCY core 70
10.2. Design Constraints (SDC file) 71
10.3. Synthesis 71
Useful commands to achieve timing optimization in synthesis 72
Flat Flow results 73
Hierachical Flow 76
10.4. PLACE & ROUTE 78
Floorplan 78
Power Plan 80
Placement 80
CTS 82
10.5. Parasitic Extraction using StarRC 89
10.6. Fixing timing and logical DRCs using Prime time ECO flow 90
10.7. Flat vs Hierarchical flows Quality of Results 92
10.8. Conclusion 98
10.9. References 99

Figure 1 PULPino Overview 10
Figure 2 zero-riscy core overview 11
Figure 3 RISCY core overview 13
Figure 4 Flowchart of Synthesis Process 15
Figure 5 physical design workflow 16
Figure 6 Chip Floorplanning 31
Figure 7 Levels of Power Distribution 38
Figure 8 placement 42
Figure 9 Coarse vs Detail Placement 43
Figure 10 Global congestion map 44
Figure 11 CTS 50
Figure 12 CTS parameters 51
Figure 13 Buffer insertion 52
Figure 14 STA flow using primetime 62
Figure 15 PrimeTime inputs and outputs 64
Figure 16 Timing Analysis Flow in PrimeTime 65
Figure 17 RISCY core 71
Figure 18 clock report 73
Figure 19 Area report 74
Figure 20 DC setup timing report of critical path 75
Figure 21 Hierarchical flow area report 76
Figure 22 DC Hierarchical flow setup report 77
Figure 23 post floor planning and early placement 79
Figure 24 power mesh of the PULPINO PULPIno SoC 80
Figure 25 PULPINO SoC after timing driven placement 81
Figure 26 PULPINO SoC Clock tree 83
Figure 27 PULPINO SoC post CTS utilization 84
Figure 28 PULPINO SoC clock area report 84
Figure 29 timing report post CTS very small hold violation to be fixed with primetime ECO flow 85
Figure 30 PULPINO SoC LVS clean 87
Figure 31 zroute DRCs report 87
Figure 32 finished chip no physical DRCs 88
Figure 33 legalized design after chip finishing 88
Figure 34 StarRC flow 89

Figure 35:report constraints pre-ECO 90
Figure 36: paths slack pre Timing fixing 90
Figure 37 Paths slack after ECO fixing 91
Figure 38 report constraints after ECO fixing 91
Figure 39 Flat Flow Fast time corner..... 92
Figure 40 Flat Flow slow corner..... 93
Figure 41 Flat flow area 94
Figure 42 flat flow power 94
Figure 43 Flat flow constraints reports..... 95
Figure 44 Flat flow Utilization..... 95
Figure 45 Hierarchical timing..... 96
Figure 46 Hierchical Area 96
Figure 47 Hierchical Power 97
Figure 48 Hierarchical constraint report 97
Figure 49 Hierarchical Utilization..... 98

1.1 Introduction

1.1.1 PULP-Platform:

The PULP (Parallel Ultra-Low Power) Platform is a state-of-the-art open-source hardware and software ecosystem designed to facilitate energy-efficient computing for a wide range of applications. Originating from a collaborative effort between ETH Zurich and the University of Bologna, the PULP Platform addresses the growing demand for high-performance, energy-efficient computing in the era of IoT, AI, and embedded systems.

The platform's modular architecture features highly configurable RISC-V based processors, optimized for low-power consumption without compromising on computational capabilities. By leveraging parallel processing and advanced power management techniques, PULP ensures optimal performance for tasks requiring intensive data processing, while maintaining minimal energy usage.

PULP provides a comprehensive suite of tools and resources, including customizable IP blocks, development boards, and a robust software stack, making it an ideal choice for researchers, developers, and companies aiming to innovate in low-power computing. With its emphasis on open-source collaboration, the PULP Platform fosters a vibrant community dedicated to pushing the boundaries of energy-efficient technology.[2]

1.1.2 What about The PULPino Project:

PULPino is a notable sub-project within the broader PULP (Parallel Ultra-Low Power) Platform ecosystem, specifically aimed at creating a lightweight, energy-efficient microcontroller system. Developed as an open-source initiative by the teams at ETH Zurich and the University of Bologna, PULPino serves as an entry point for developers and researchers interested in exploring ultra-low power processing and the capabilities of the RISC-V architecture.[1]

PULPino features a single-core RISC-V processor, designed for simplicity and low power consumption, making it ideal for IoT applications, wearable devices, and other embedded systems requiring efficient, small-footprint processing. The microcontroller includes essential peripherals and interfaces, allowing it to handle a variety of sensor inputs and communication protocols.

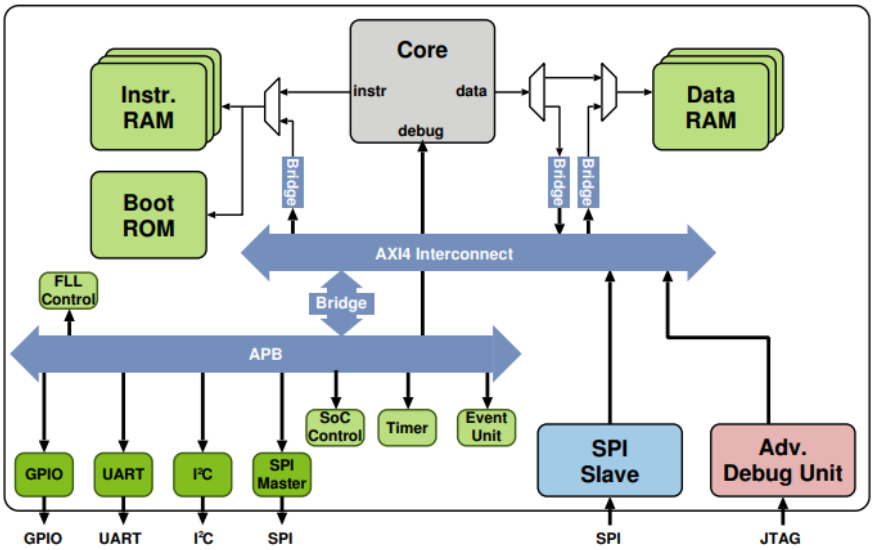

Figure 1 PULPino Overview

The PULPino project provides a comprehensive development environment, including hardware description files, simulation models, and a complete software development kit (SDK). This makes it accessible for experimentation and customization, enabling users to tailor the system to their specific needs. The project's open-source nature promotes community contributions and collaborative advancements, further enhancing its functionality and application range.

By offering a practical and efficient solution for ultra-low power computing, PULPino embodies the PULP Platform's commitment to advancing energy-efficient technology and democratizing access to innovative hardware and software resources.[1]

1.1.3 Ibex (Zero-riscy) Core for PULPino Project:

The Ibex (Zero-riscy) core is an integral part of the PULPino project, embodying the principles of energy efficiency and simplicity central to the PULP Platform's mission. Zero-riscy is a lightweight, 32-bit RISC-V processor core with IMC Instruction Set Architecture (ISA) designed to deliver optimal performance with minimal power consumption, making it ideal for low-power applications such

as IoT devices, wearable technology, and embedded systems. Ibex has a 2-stage pipeline, the 2 stages are:

1.1.4 Key Features of the Ibex (Zero-riscy) Core:

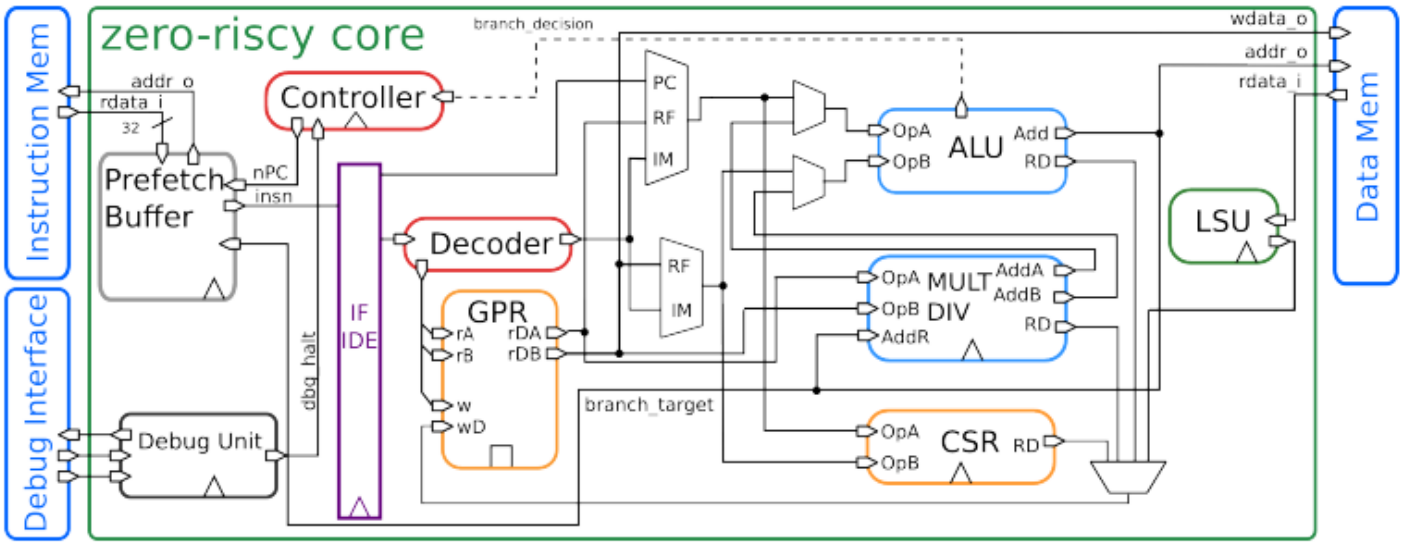

Figure 2 zero-riscy core overview

Instruction Fetch (IF):

Fetches instructions from memory via a prefetch buffer, capable of fetching 1 instruction per cycle if the instruction side memory system allows.

Instruction Decode and Execute (ID/EX):

Decodes fetched instruction and immediately executes it, register read and write all occurs in this stage. Multi-cycle instructions will stall this stage until they are complete.

All instructions require two cycles minimum to pass down the pipeline. One cycle in the IF stage and one in the ID/EX stage. Not all instructions can be completed in the ID/EX stage in one cycle so will stall there until they complete. This means the maximum IPC (Instructions per Cycle) Ibex can achieve is 1 when multi-cycle instructions aren't used.

Zero-riscy is a configuration of Ibex with IMC instruction set, where (I) stands for Integer Computational, (M) stands for Standard Extension for Integer Multiplication and Division, and (C) stands for Standard Extension for Compressed Instructions. Zero-riscy leverages the open RISC-V instruction set

architecture (ISA), which provides flexibility and extensibility while being free from proprietary restrictions. This openness allows for broad customization and optimization.

The core is engineered to operate with very low power consumption, aligning with the PULPino project's goal of creating ultra-low power microcontroller systems. This makes it suitable for battery-powered and energy-harvesting applications.

Zero-riscy is designed with a compact implementation, ensuring it requires minimal silicon area. This small footprint is advantageous for integrating the core into systems with stringent space constraints.

Zero-riscy can be easily integrated with other components and peripherals within the PULPino ecosystem, providing a versatile foundation for developing complete low-power systems.[1]

1.1.5 Applications of Pulpino with Ibex (Zero-riscy) core:

Internet of Things (IoT): Providing the computational power needed for sensors, data aggregation, and communication while maintaining long battery life.

Embedded Systems: Offering a reliable and efficient processing unit for various embedded applications, from consumer electronics to industrial automation.[3]

1.1.6 RI5CY Core for PULPino Project:

The RI5CY core is another crucial component within the PULPino project, providing a more feature-rich and higher-performance alternative to the Zero-riscy core. Developed as part of the PULP (Parallel Ultra-Low Power) Platform, RI5CY is a 32-bit RISC-V processor core designed to deliver an excellent balance of performance and energy efficiency. It is well-suited for applications requiring more computational power while still adhering to the low-power ethos of the PULP Platform.[2]

1.1.7Key Features of the RI5CY Core

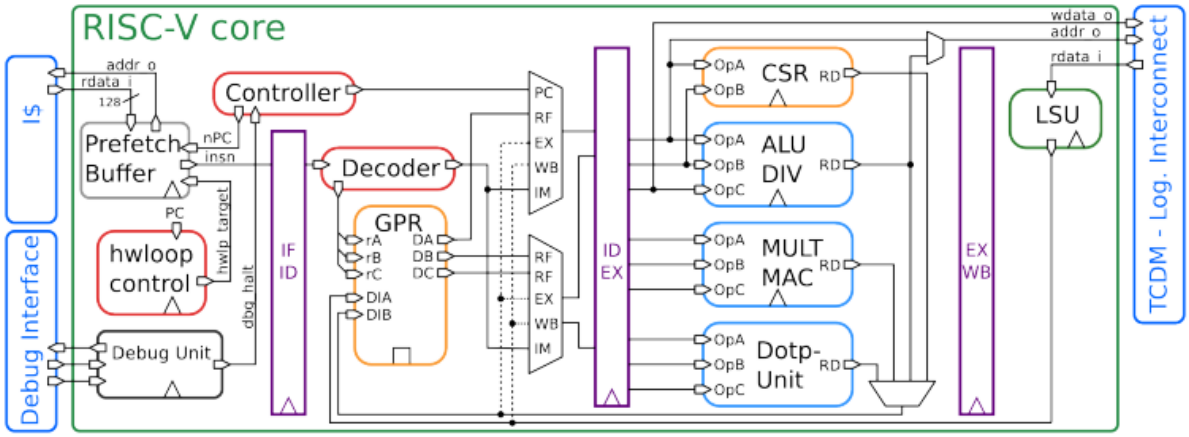

Figure 3 RISCY core overview

Like Zero-riscy, the RI5CY core is built on the open RISC-V instruction set architecture (ISA). This provides the flexibility to customize and extend the core as needed, leveraging the benefits of an open-source ecosystem.

RI5CY offers higher performance compared to Zero-riscy, thanks to its four-stage pipeline, hardware multiply/divide units, and support for DSP (Digital Signal Processing) extensions. This makes it capable of handling more demanding computational tasks. Despite its enhanced capabilities, RI5CY is designed to be energy-efficient, ensuring that it can be used in power-constrained environments such as IoT devices and embedded systems,

also the inclusion of DSP extensions allows RI5CY to efficiently execute complex signal processing tasks, making it ideal for applications in areas like audio processing, communication, and sensor data analysis.

Customizable and Extensible: The open-source nature of RI5CY allows developers to tailor the core to specific application requirements. This includes adding custom instructions or hardware accelerators to further boost performance for particular tasks.

1.1.8 Applications of RI5CY

Internet of Things (IoT): Providing the necessary computational power for advanced data processing, communication, and control tasks in IoT devices.

Wearable Technology: Enabling complex features and functionalities in wearable devices while maintaining low power consumption.

Embedded Systems: Offering a robust and efficient processing unit for industrial, automotive, and consumer electronics applications.

Digital Signal Processing: Performing efficient signal processing for applications in audio, video, and communications.

1.1.9 The project vision:

Implementation of the PULPino SoC with RISCY core, targeting IoT and wireless charging applications leveraging RSICY efficient power consumption and high performance with different techniques of implementation.

Trying different synthesis flows using Design Compiler (DC) then going through the physical implementation flow using IC Compiler II to decide which is the most suitable flow to achieve power optimized implementation. The project will include the use of StarRC parasitic extraction tool and Primetime signoff tool to ensure accurate timing closure and DRCs clean implementation.

2.1 Introduction to Synthesis and Design Compiler

Design Compiler is a widely used tool developed by Synopsys for the synthesis of digital circuits. Synthesis is a critical step in the digital design flow, transforming high-level design descriptions into optimized gate-level representations. This process is essential for the creation of reliable, efficient, and manufacturable semiconductor devices.

2.1.1 Key Concepts in Synthesis

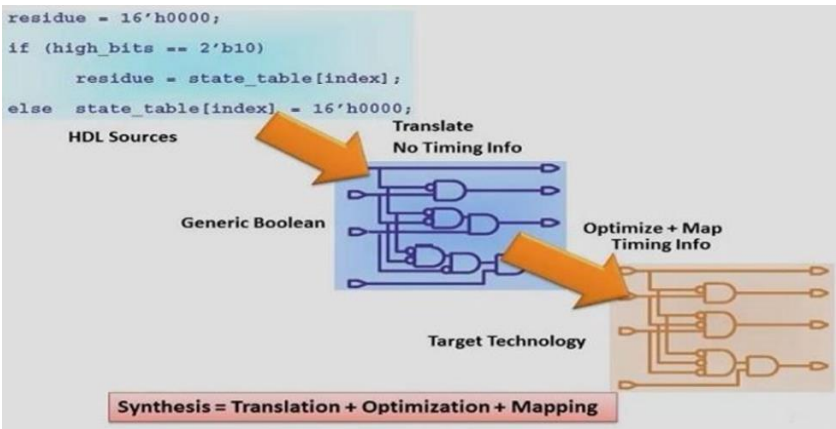

Figure 4 Flowchart of Synthesis Process

High-Level Description: Designers typically start with a high-level description of the digital circuit, often written in hardware description languages (HDLs) like VHDL or Verilog. These descriptions focus on functionality rather than physical implementation.

Technology Library: A technology library contains information about the available standard cells and their characteristics, such as timing, power, and area. This library is specific to the manufacturing process technology used for the chip; in our project we will use SAED 14nm technology.

Logical Synthesis: This process involves converting the high-level HDL description into an intermediate representation, typically a Register Transfer Level (RTL) description. From the RTL, the synthesis tool generates a gate-level netlist, mapping the logic to specific standard cells from the technology library.

Optimization: During synthesis, the Design Compiler performs various optimizations to meet design constraints. These constraints can include timing

(speed), area (size), power consumption, and other performance metrics. The goal is to create a design that meets or exceeds the specified requirements.

Verification: Post-synthesis verification is crucial to ensure that the synthesized design correctly implements the intended functionality and meets the required performance specifications. This step often involves simulation and formal verification techniques.[4]

2.1.2 Typical Synthesis Workflow

Design Entry: Write the HDL code describing the design's functionality.

Constraint Definition: Define design constraints, such as timing, area, and power targets.

Synthesis: Use the Design Compiler to synthesize the HDL code into a gate-level netlist.

Optimization: Apply various optimization techniques to meet the design constraints.

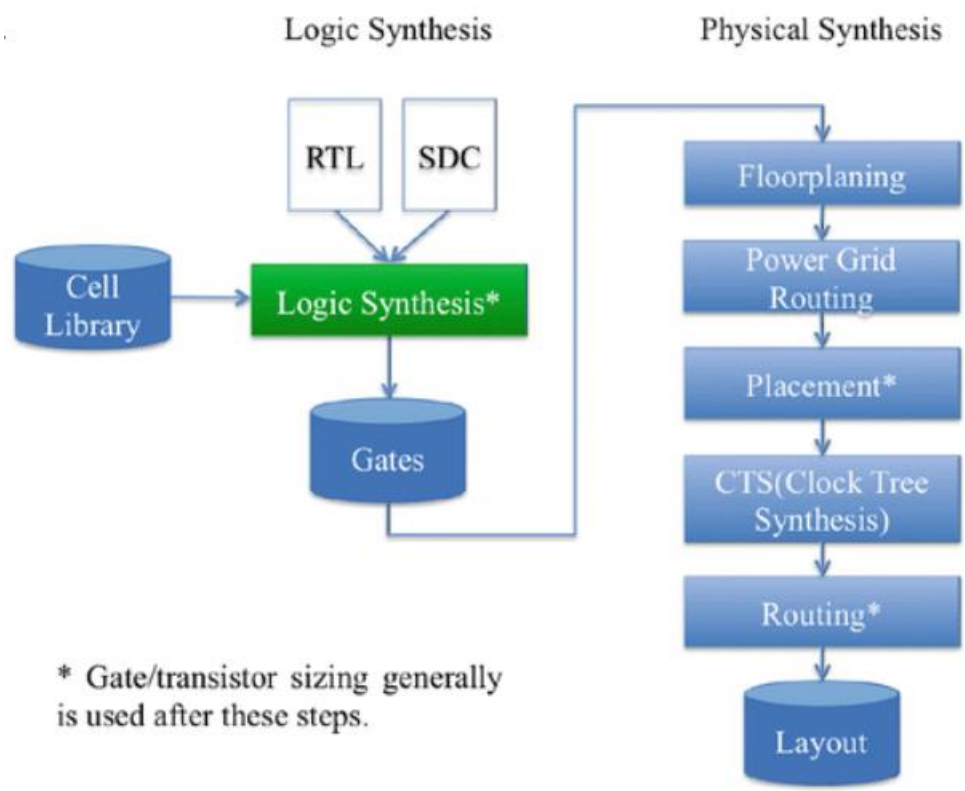

Figure 5 physical design workflow

Verification: Perform functional and timing verification to ensure the synthesized design meets all specifications.

Output: Generate the final gate-level netlist and associated reports for further steps in the design flow, such as placement and routing.[4]

2.1.3 Importance of Synthesis in Digital Design

Synthesis bridges the gap between high-level design and physical implementation. It ensures that the design can be manufactured efficiently while meeting all performance requirements. By automating this process, tools like Design Compiler significantly reduce the time and effort required to bring a digital design from concept to silicon.

2.1.4 Design Compiler: Features and Workflow

Design Compiler offers a robust set of features to support the synthesis process:

RTL Analysis and Optimization: Analyzes the RTL code for potential issues and performs optimizations to improve the design's quality.

Technology Mapping: Maps the optimized RTL to the cells available in the technology library.

Constraint Management: Allows designers to specify and manage design constraints, ensuring that the synthesized design meets the necessary requirements.

Incremental Synthesis: Supports incremental changes, allowing designers to make small adjustments without re-synthesizing the entire design.

Power Optimization: Includes techniques for reducing power consumption, such as clock gating and multi-threshold CMOS (MTCMOS) strategies.

Area optimization: focuses on reducing the physical space occupied by the synthesized circuit on the silicon die. This is crucial for lowering manufacturing costs, increasing yield, and improving performance.

Timing optimization: ensures that the design meets the required clock speed and timing constraints, which include setup time, hold time, and clock skew.

2.1.5 Design compiler different flows

"Flat," and "Hierarchical" are different approaches for handling the design during synthesis. Each approach has its own benefits and challenges, depending on the complexity and size of the design, as well as the design goals.

2.1.6 Flat Flow

Flat Flow involves synthesizing the entire design as a single entity without considering its internal hierarchical structure.

Characteristics of flat flow:

Single Level: The entire design is treated as one large block.

Simple Setup: Easier to set up because there's no need to manage multiple hierarchical levels.

Resource Intensive: Requires more computational resources (memory and CPU) since the entire design is synthesized at once.

Advantages of flat flow:

Optimal Optimization: Optimized design is synthesized together; it allows for more global optimizations across the whole design.

Simplified Analysis: Timing analysis and other checks are simpler because they are performed on a single, unified netlist.

Disadvantages of flat flow:

Scalability Issues: Not suitable for very large designs due to high memory and CPU requirements.

Longer Runtime: Larger designs can take significantly longer to synthesize.

2.1.7 Hierarchical Flow

Hierarchical Flow synthesizes each module or block of the design separately before integrating them into the top-level design.

Characteristics of hierarchical flow:

Multiple Levels: The design is divided into smaller blocks, which are synthesized independently.

Modular Approach: Allows for a modular design methodology, where each block can be optimized separately.

Easier Management: Easier to manage changes and incremental synthesis since changes in one block do not necessarily require re-synthesizing the entire design.

Advantages of hierarchical flow:

Scalability: More scalable for large designs since each block can be synthesized independently.

Resource Management: Reduces the peak memory and CPU usage because each block is smaller than the entire design.

Incremental Synthesis: Faster turnaround time for changes since only the modified blocks need to be re-synthesized.

Disadvantages of hierarchical flow:

Suboptimal Optimization: May miss some global optimizations because each block is optimized independently.

Complexity in Integration: Requires careful management of the interfaces between blocks to ensure overall design performance.

Choosing the appropriate synthesis flow depends on the design size, complexity, resource availability, and the desired accuracy of pre- and post-layout correlation. Flat flow is simpler but less scalable, and hierarchical flow offers better management for large designs.

2.1.8 SAED 14nm technology

The SAED_EDK14_FINFET_CORE Digital Standard Cell Library is built using SAED14nm FinFET 1P9M 0.8V/1.5V/1.8V design rules. The library is created aimed at optimizing the main characteristics of designed integrated circuits by its help. The library includes typical miscellaneous combinational and sequential logic cells for different drive strengths. Besides, the library contains all the cells which are required for different styles of low power (multi-voltage,

power gating, clock gating, multi-threshold, etc.) designs. Those are the following: Isolation Cells, Level Shifters, Retention Flip-Flops, Always-on Buffers and Power Gating Cells. In order to implement multi-threshold low power techniques HighVt(HVT), Low-Vt(LVT), Super Low-Vt(SLVT) and Regular-Vt(RVT) versions of the Library will be created. The presence of all these cells will provide the support of integrated circuits design with different core voltages to minimize dynamic and leakage power.

2.1.9 Synthesis Commands and Explanations:

2.1.9.1 Constrains commands and explanation:

DC Variables

dc_shell > set_fix_multiple_port_nets -all -buffer_constants -feedthroughs

Purpose: Ensures that nets with multiple drivers are handled correctly by inserting buffers and feedthroughs.

Expected Output: No “assign” statements in the generated netlist, and correct handling of multiple driver nets.

dc_shell > define_name_rules no_case -case_insensitive

Purpose: Defines a naming rule to make names case insensitive.

Expected Output: All names in the design are treated case-insensitively.

dc_shell > change_names -rule no_case -hierarchy

Purpose: Applies the previously defined case-insensitive naming rule to the entire design hierarchy.

Expected Output: Consistent case-insensitive naming throughout the design.

dc_shell > change_names -rule verilog -hierarchy

Purpose: Applies the Verilog naming conventions to the entire design hierarchy.

Expected Output: Names in the design conform to Verilog naming conventions.

dc_shell > set verilogout_no_tri true

Purpose: Ensures no tri-state buffers are generated in the Verilog output.

Expected Output: Tri-state buffers are avoided in the netlist.

dc_shell > set verilogout_equation false

Purpose: Disables the generation of continuous assignment equations in the Verilog output.

Expected Output: No continuous assignment equations in the netlist.

dc_shell > set_fix_multiple_port_nets -all -buffer_constants [get_designs *]

Purpose: Consistency in handling multiple port nets for all designs.

Section 1: Clock Definition

dc_shell > set CLK_NAME .....

dc_shell > set CLK_PER .....

dc_shell > set CLK_SETUP_SKEW .....

dc_shell > set CLK_HOLD_SKEW .....

dc_shell > set CLK_LAT .....

dc_shell > set CLK_RISE .....

dc_shell > set CLK_FALL .....

Purpose: Defines clock names, periods, setup and hold skew, and clock latencies, rise, and fall times.

dc_shell > create_clock -name $CLK_NAME -period $CLK_PER -waveform "0 [expr $CLK1_PER/2]" [get_ports clk]

Purpose: Master clocks are defined in design with specified names, periods, and waveforms.

dc_shell > set_dont_touch_network [get_clocks "....."]

Purpose: Prevents optimization changes to the specified clock networks.

Expected Output: Clock networks are preserved during optimization.

```
dc_shell > create_generated_clock -master_clock .... -source [get_ports ....] \
-name "............" [get_port ......] \
-invert -divide_by 1
```

Purpose: Creates generated clocks based on the master clocks with inversion and no division.

```
dc_shell > set_dont_touch_network [get_clocks "........."]
```

Purpose: Prevents optimization changes to the generated clock networks.

```
dc_shell > set_clock_uncertainty -setup $CLK_SETUP_SKEW [all_clocks]
dc_shell > set_clock_uncertainty -hold $CLK_HOLD_SKEW [all_clocks]
dc_shell > set_clock_transition -rise $CLK_RISE [all_clocks]
dc_shell > set_clock_transition -fall $CLK_FALL [all_clocks]
dc_shell > set_clock_latency $CLK_LAT [all_clocks]
```

Purpose: Sets clock uncertainties, transitions, and latencies for all clocks.

Expected Output: Defined timing constraints for setup, hold, transition, and latency.

```
dc_shell > report_clocks
```

Purpose: Generates a detailed report listing all the defined clocks and their properties.

Section 2: Clocks Relationship

```
dc_shell > set_clock_groups -asynchronous \
        -group {.......} \
        -group {.......} \
        -group {.......}
```

Purpose: Defines asynchronous clock groups to specify that these clocks have no timing relationship and defining the path between them as a false path.

Expected Output: Properly grouped asynchronous clocks for timing analysis.

Section 3: Set Input/Output Delay on Ports

dc_shell > set synth_outputs [list {output_name it's delay}]

Purpose: Defines output delays for specific ports as a percentage of the clock.

dc_shell > set synth_inputs [list {input_name it's delay}]

Purpose: Defines input delays for specific ports as a percentage of the clock.

Expected Output: Timing constraints for input and output ports based on the specified delays.

Section 4: Driving Cells

dc_shell > set_driving_cell -library ..... -lib_cell ..... -pin ... [all_inputs]

Purpose: Specifies the driving cells for all input ports using the specified library cell with a specific size and pin.

Expected Output: Properly driven input ports with the specified library cell.

Section 5: Output Load

dc_shell > set max_cap .....

dc_shell > set min_cap .....

dc_shell > set_load -max $max_cap [all_outputs]

dc_shell > set_load -min $min_cap [all_outputs]

dc_shell > set_max_fanout ..... [current_design]

Purpose: Sets the maximum and minimum capacitive loads and maximum fanout for all output ports.

Expected Output: Defined output load constraints for timing and synthesis.

Section 6: Operating Condition

dc_shell > set_operating_conditions -min_library "....." -min "....." -max_library "....." -max "....."

Purpose: Specifies the operating conditions for minimum and maximum timing analysis using the given libraries and corners.

Expected Output: Defined operating conditions for timing analysis.

dc_shell > set_wire_load_model -name ..... -library .....

dc_shell > set_case_analysis 0 [get_ports .....]

Purpose: Sets the wire load model to be used during synthesis and sets a case analysis condition, likely indicating that the test mode input is set to 0.

Expected Output: The design will be analyzed with a defined wire load model and for synthesis the test mode input will be set to 0.

2.1.9.2 Synthesis setup commands and explanation:

dc_shell > set_host_options -max_cores 1

Purpose: The Design Compiler will use only one core during execution, to ensure consistency in timing or resource usage.

dc_shell > set top_module pulpino_top

Purpose: Specifies the top-level module for the design, which will be used as the entry point for synthesis and other design operations.

dc_shell > define_design_lib work -path ./work

Purpose: Defines the working library named work and sets its path to ./work, where intermediate files and compiled design data will be stored.

dc_shell > set_svf $top_module.svf

Purpose: The SVF file pulpino_top.svf is created, which will be used for formal verification.

dc_shell > lappend search_path “standard cell lib direction”

dc_shell > lappend search_path “RTL design direction”

Purpose: Adding the specified directories to the search_path variable, enabling the Design Compiler to locate the RTL design and library for standard cells (.db) files.

dc_shell > set SSLIB "saed14...vt_ss0p72vm40c.db"

dc_shell > set TTLIB "saed14...vt_tt0p8v25c.db"

dc_shell > set FFLIB "saed14...vt_ff0p88v125c.db"

Purpose: Defines variables for the paths to the standard cell library files, slow corner and typical corner and fast corner, for different operating conditions like temperature and voltage.

dc_shell > set target_library [list $SSLIB $TTLIB $FFLIB]

dc_shell > set link_library [list * $SSLIB $TTLIB $FFLIB]

Purpose: The target_library variable is set to include the standard cell libraries for synthesis, The link_library variable is set to include all necessary library files for linking the design, since we do not have any macros or IPs, we will find that the target library is the same as the link library.

dc_shell > source ./dont_use_generic.tcl

Purpose: Sources a TCL script that specifies certain cells to be excluded from use during synthesis.

dc_shell > source ./elaborate_PULPino.tcl

Purpose: Sources a TCL script that performs the elaboration of the top module pulpino_top.

2.1.9.3 Elaboration commands and explanation:

dc_shell > analyze -autoread $RTL_files

Purpose: Analyzes the RTL files listed in RTL_files for syntax errors and reads them into the Design Compiler's database.

dc_shell > elaborate pulpino_top

Purpose: Elaborates the design starting from the top module pulpino_top. This process resolves all instances and hierarchical references in the design.

dc_shell > link

Purpose: Resolves any remaining unresolved references in the design.

Expected Output: All references in the design are resolved, and the design is fully linked.

2.1.9.4 Finalizing the synthesis

dc_shell > current_design $top_module

Purpose: The current design context is switched to the top module.

dc_shell > check_design > design_checks.log

Purpose: Checks the design for consistency issues and logs the results.

Expected Output: A log file (design_checks.log) is created containing details about any design inconsistencies.

dc_shell > group_path -name input -from [all_inputs]

dc_shell > group_path -name outputs -to [all_outputs]

dc_shell > group_path -name comb -from [all_inputs] -to [all_outputs]

Purpose: Groups paths for timing analysis.

input: Paths starting from all input ports.

outputs: Paths ending at all output ports.

comb: Combinational paths from all inputs to all outputs.

Expected Output: Paths are grouped for better management during timing analysis.

dc_shell > source -verbose ./cons.tcl

Purpose: Sources an external TCL script containing design constraints, constraints specified in the sourced script are applied to the current design.[5]

2.1.9.5 Compile the design

dc_shell > compile_ultra

The compile_ultra command in Synopsys Design Compiler is an advanced synthesis command used to optimize a design for timing, area, and power. This

command integrates multiple optimization techniques, including the use of advanced algorithms for gate-level optimization, clock gating, and retiming.

2.1.9.6 Detailed explanation of the compile_ultra options

-gate_clock: This option enables the synthesis tool to insert clock gating logic, which reduces dynamic power consumption by turning off the clock to portions of the circuit when they are not in use. Clock gating is a widely used technique for reducing power consumption in synchronous digital circuits.

-no_seq_output_inversion: This option prevents the synthesis tool from using sequential cells with inverted outputs. By default, the synthesis tool might use sequential cells with inverted outputs to meet timing constraints, but this option ensures that only non-inverted outputs are used, which can simplify timing analysis and design debugging.

-retime: This option enables retiming, an optimization technique that moves registers across combinational logic to improve the timing performance of the design. Retiming can help achieve better clock frequencies by balancing the path delays between registers.

-incremental: This option is used after the initial compilation to make further optimizations without starting from scratch. It allows the tool to refine the design iteratively, making small adjustments to improve performance, area, or power.

Expected Output:

Optimized Netlist: The primary output is an optimized gate-level netlist that meets the specified constraints for timing, area, and power. The netlist is ready for further verification, including timing analysis and functional verification.

Timing Reports: Detailed reports on the timing performance of the design, including setup and hold times, slack, and critical paths.

Area Reports: Information on the total cell area used in the design, providing insights into the efficiency of the synthesis process.

Power Reports: Estimates of the power consumption of the design, including dynamic and static power components.[4]

2.1.9.7“compile_ultra” Command with “-no_autoungroup” Option

The -no_autoungroup option is one of the various options available with the compile_ultra command. Understanding this option requires an understanding of how hierarchy is handled during synthesis.

Hierarchy Handling in Synthesis

When synthesizing a design, Design Compiler can either preserve the hierarchy of the RTL design or flatten it. Flattening the hierarchy allows for more aggressive optimizations across module boundaries, but it can make the design harder to understand and debug. On the other hand, preserving hierarchy can make it easier to maintain and debug the design but might limit some optimization opportunities.

compile_ultra -no_autoungroup

The -no_autoungroup option tells the compiler not to automatically ungroup certain hierarchical levels during synthesis. Ungrouping refers to the process of flattening the hierarchy, which can expose more optimization opportunities by allowing the tool to see and optimize across boundaries that would otherwise be hidden.

dc_shell > set_svf -off

Purpose: Disables the generation of the Synopsys Verification Format (SVF) file.

2.1.9.8 Write Out Files

dc_shell > define_name_rules no_case -case_insensitive

dc_shell > change_names -rule no_case -hierarchy

dc_shell > change_names -rule sverilog -hierarchy

dc_shell > set verilogout_no_tri true

dc_shell > set verilogout_equation false

dc_shell > write_file -format verilog -hierarchy -output Pulpino_netlist.v

dc_shell > write_file -format ddc -hierarchy -output pulpino.ddc

dc_shell > write_sdf pulpino.sdf

dc_shell > write_sdc -nosplit pulpino.sdc

Purpose: Applies naming rules and writes out various files.

- define_name_rules no_case -case_insensitive: Defines naming rules to be case insensitive.
- change_names -rule no_case -hierarchy: Changes names according to the no_case rule throughout the hierarchy.
- change_names -rule sverilog -hierarchy: Changes names according to the systemverilog rule throughout the hierarchy.
- set verilogout_no_tri true: Prevents the use of tri-state buffers in the Verilog output.
- set verilogout_equation false: Disables equation generation in the Verilog output.
- write_file -format verilog -hierarchy -output pulpino_netlist.v: Writes the synthesized design to a Verilog file.
- write_file -format ddc -hierarchy -output pulpino.ddc: Writes the design to a Design Compiler database file.
- write_sdf pulpino.sdf: Writes the Standard Delay Format file.
- write_sdc -nosplit pulpino.sdc: Writes the Synopsys Design Constraints file.

3.1 IC Compiler II:

The Synopsys IC Compiler II provides a complete netlist-to-GDSII design solution, which includes innovations for flat and hierarchical design planning, early design exploration, congestion aware placement and optimization, clock tree synthesis, advanced node routing convergence, manufacturing compliance, and signoff closure.[1]

ICC2 makes use of cutting-edge algorithms, optimization techniques, and simulation capabilities to improve performance, reduce power consumption, and save space. ICC2 provides industry standard compliance by utilizing unique algorithms and optimization approaches.[4][11]

Synopsys IC Compiler II's infrastructure has been built to handle increasingly large designs using a highly scalable data model that enables high capacity, engine efficiency and broad parallelism. And It's library manager produces a common reference library that integrates the physical, logical and timing data in a compact, fast-access database.[4]

ICC2 has comprehensive verification tools such as functional and physical verification using simulation and formal analysis, as well as design rule checking and layout versus schematic (LVS) tests. This guarantees error-free designs and consistent circuit functioning.[4]

With a modern infrastructure, new, patented techniques for design planning, optimization and clocking, ICC2 delivers an order of magnitude productivity improvement over current solutions.[4][11]

3.2 overview on PnR flow:

3.2.1 Floorplanning

Floorplanning involves the strategic determination of the chip's major design objects' size and placement. Key influencers include the digital frontend, digital backend, analog-mixed signal designer, and the project manager. Decisions made during floorplanning significantly impact the subsequent stages of the design flow. [3]

3.2.2 Power Planning & Boundary Cell Insertion

Strategic power planning ensures efficient power distribution, and boundary cell insertion establishes the chip's periphery, shaping its overall structure.

3.2.3 Placement

Placement involves positioning standard cells within the defined floorplan, considering factors such as congestion, cell density, and overall accessibility for routing. [3]

3.2.4 CTS and Post-CTS Logical Optimization

Clock Tree Synthesis (CTS) ensures a balanced and efficient distribution of clock signals throughout the design. Post-CTS logical optimization refines the design for improved performance.

3.2.5 Routing

The routing stage involves the creation of physical interconnections between the placed cells, adhering to design rules and ensuring the integrity of signal paths.

3.2.6 PrimeTime

PrimeTime breaks a design down into timing paths, calculates the signal propagation delay along each path, and checks for violations of timing constraints inside the design and at the input/output interface.

3.3 Floorplanning Overview:

Floorplanning is the process of partitioning logical blocks into physical blocks, sizing and placing the blocks and performing a floorplan-level placement of macros and standard cells. The goal of floorplanning is to increase the efficiency of downstream physical design steps to enable a robust, optimized design. Floorplanning can also be an iterative process that reshapes blocks, creates a new cell placement, reallocates timing budgets, and rechecks top-level timing until an optimal floorplan is created. [4]

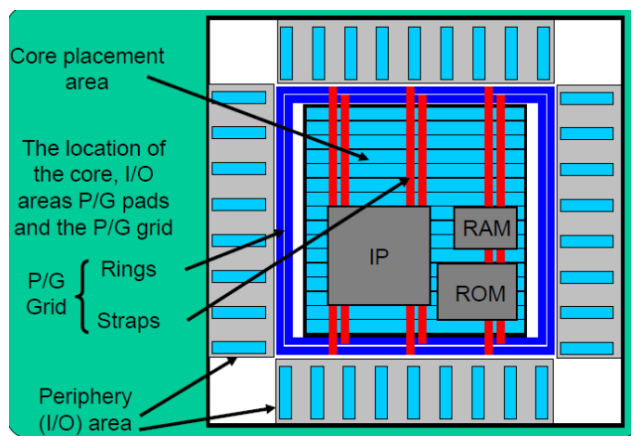

Figure 6 Chip Floorplanning

In this step, the preferred routing directions of the layers are defined, the pins are placed as well, it's preferred to alternate between horizontal and vertical routing directions to allow the best utilization and better congestion results. Also, the routing is limited to a set of layers as to tell the tool that it's only allowed to route over the available layers physically.

The chip floorplanning process involves several considerations:

3.3.1 Metal Layers Definition

Defining the direction of each metal layer in the design.

3.3.2 IP Floorplanning

For Intellectual Property (IP) floorplanning, the focus shifts to die area, core area, pin placement and power delivery. [6]

3.3.3 TAP Cells

Well tap cells (or Tap cells) are used to prevent the latch-up issue in the CMOS design. Well tap cells connect the nwell to VDD and p-substrate to VSS in order to prevent the latch-up issue.

3.3.4 Standard cell placement and Routing

Standard cell placement involves creating site rows and placing cells strategically to optimize routing and power grid considerations and have a good area utilization to make it easy to close routing. [6]

3.3.5 Guidelines for a Good Floorplan

Some guidelines for a well-structured floorplan are emphasizing macro placement, routing channels, pin accessibility, and congestion avoidance. [6]

3.4 Floorplanning Commands:

Before the floorplanning stage, we should first create the library which will be used in the PnR flow.

The first step in library creation is set search_path, set target_library and set link_library as in the synthesis stage and enter all the flavors which will be used in the PnR flow.

```
set_app_var link_library $link_library
```

then this command will be used (it sets the specified application variable. This command sets the variable to its default value or to a new value you specify).

After that creating the library by the following command:

create_lib -technology $DESIGN_REF_TECH_PATH/saed14nm_1p9m_mw.tf -ref_libs $NDM_REFERENCE_LIB_DIRS ${DESIGN_NAME}

This command creates a new design library. The library contain technology information specified with the -technology option and specify the library's reference library path list with the -ref_libs option.

The final step is using the following command to read in parasitic model data files. After the data is read in, it is stored in current library and the TLUPlus files or common tech file are no longer referenced.

- source /mnt/hgfs/Project/ICC2/scripts/fp/mcmm.tcl

MCMM file takes more scenarios to give the most accurate results, however, in this case only 2 scenarios were introduced. The inputs to the mcmm file are the comstraint files, map file, the Tlup_max and Tlup_min

3.4.1The MCMM.tcl file:

```
remove_corners -all
remove_modes -all
remove_scenarios -all

create_corner slow
create_corner fast

read_parasitic_tech \
        -tlup $TLUPLUS_MAX_FILE \
        -layermap $MAP_FILE \
        -name tlup_max

read_parasitic_tech \
        -tlup $TLUPLUS_MIN_FILE \
        -layermap $MAP_FILE \
        -name tlup_min

set_parasitics_parameters \
        -early_spec tlup_min \
        -late_spec tlup_min \
        -corners {fast}
```

```
set_parasitics_parameters \
        -early_spec tlup_max \
        -late_spec tlup_max \
        -corners {slow}

create_mode func

### FUNC_FAST
current_corner fast
source $fast_corner
create_scenario -mode func -corner fast -name func_fast
read_sdc $Constraints_file

### FUNC_SLOW
current_corner slow
source $slow_corner
create_scenario -mode func -corner slow -name func_slow
read_sdc $Constraints_file
```

- sh rm -rf $DESIGN

This command is used before creating the library to delete any library which be created before to create a new one.

- read_verilog -top $DESIGN $gate_verilog

This command reads one or more structural, gate-level Verilog netlists into the tool.

-top option: Specifies the name of the top module in the design hierarchy.

current_design $DESIGN

This command sets the current block to the block containing a specified top-level logic module, and returns that block as a collection.

- read_sdc ${SDC_PATH}/${DESIGN}.sdc

This command reads in a script file in Synopsys Design Constraints (SDC) format. The SDC file is an output of the synthesis and it contains the constraints which is used.

- set_attribute [get_layers M1] routing_direction vertical
- set_attribute [get_layers M2] routing_direction horizontal

- set_attribute [get_layers M3] routing_direction vertical
- set_attribute [get_layers M4] routing_direction horizontal
- set_attribute [get_layers M5] routing_direction vertical
- set_attribute [get_layers M6] routing_direction horizontal
- set_attribute [get_layers M7] routing_direction vertical
- set_attribute [get_layers M8] routing_direction horizontal
- set_attribute [get_layers M9] routing_direction vertical
- set_attribute [get_layers MRDL] routing_direction horizontal

This command sets values to specified value on any specified list of objects, used in this case to define the direction of each layer.

- initialize_floorplan \

  -core_utilization 0.75 \

  -flip_first_row true \

  -core_offset {14.4 14.4 14.4 14.4}

This command creates the initial floorplan with a die boundary, core, site arrays and rows, and wire tracks, in this case, it takes the utilization ratio of the core (core area to chip area ratio), as well as the offset of the core from the borders of the chip, vertically and horizontally.

-core_utilization option: Specifies the utilization of the core area. The utilization is the total area of the core occupied by all standard cells and macro cells divided by the total core area. its value is between 0 and 1. The cell area includes all standard and macro cells. For example, a core utilization of 0.8 specifies that 80 percent of the core area is used for cell placement at this stage. The tool might later add more cell area, the remaining area is available for routing.

-flip_first_row option: Specifies whether the command flips the first row at the bottom of the core area for horizontally placed cell rows, or flips the leftmost row for vertically placed cell rows.

-core_offset option: Specifies the distance between the side of the core and the side of the die boundary.

- set_block_pin_constraints -self \

  -allowed_layers {M5 M6} \

  -sides {1 2 3 4} -pin_spacing 3

This command sets pin placement constraints on blocks and it is used to put the pins on higher layers instead of metal 2 (the default).

-self option: Applies constraints only to the top-level design.

-allowed_layers option: Specifies the set of metal layers allowed for pin placement.

-sides option: Specifies the block sides on which the pin must be placed.

-pin_spacing option: Specifies the minimum number of wire tracks between adjacent pins or between a pin and a preroute wire that cuts across a block edge in the normal direction.

- place_pins -ports [get_ports *]

Places pins on blocks within the current design or on the current block.

-ports option: Specifies the top-level ports for pin assignment.

- set_attribute [get_ports *] physical_status fixed

This command makes the physical status of the ports which be placed to be fixed.

- get_attribute [get_ports *] is_fixed

This command checks if the previous command is executed correctly (physical status of all ports are fixed).

- create_tap_cells -lib_cell [get_lib_cells "saed14rvt_ss0p6vm40c/SAEDRVT14_TAPDS"] -distance 20 -pattern stagger

This command adds tap cells to the design. A tap cell is a special cell with a well tie, substrate tie, or both.

-lib_cell option: Specifies the library reference cell to be used as a tap cell. You must specify a single library cell.

-distance option: Specifies the distance in microns between two tap cells in a row.

-pattern option: Specifies the tap cell insertion pattern. (Stagger: adds tap cells to every row)

- `create_net -power  "VDD"`
- `create_net -ground "VSS"`

Creates one or more power, ground, tie, or signal nets.

- `connect_pg_net -net "VDD" [get_pins -hierarchical "*/VDD"]`
- `connect_pg_net -net "VSS" [get_pins -hierarchical "*/VSS"]`

Connects power, ground, and tie-off pins to the specified power and ground nets.

-net option: Specifies the PG net to connect.

- `create_placement -floorplan -timing_driven`

Performs coarse placement on the current design.

-floorplan option: Specifies that floorplanning placement should be used.

-timing_driven option: Enables timing-driven placement.

- `legalize_placement`

Executes detailed placement on the current design.

4.1 Powerplanning

4.1.1 powerplanning overview:

Power planning is a critical aspect of integrated circuit design that ensures a uniform supply of voltages to all standard cells and macros. This part delves into the intricacies of power planning, addressing challenges such as IR drop, dynamic IR, and electromigration. [4]

Power planning is essential for maintaining stable voltages, minimizing noise, avoiding IR drop, and preventing electromigration-related issues. The power distribution network (PDN) aims to uniformly distribute power throughout the chip while minimizing area utilization and simplifying layout. [4]

Three Levels of Power Distribution

1- **Rings**: Circulate (VDD, VSS) around the chip.
2- **Stripes**: Connect (VDD, VSS) from rings to the core.
3- **Rails**: Link (VDD, VSS) to standard cell pins.

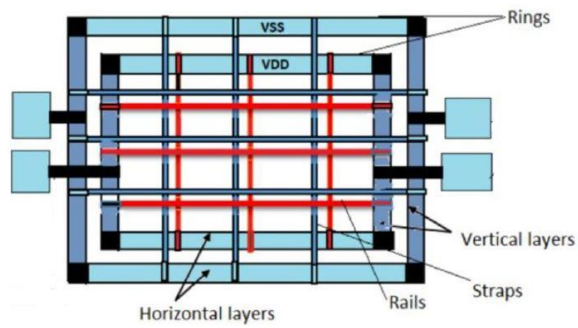

Figure 7 Levels of Power Distribution

In this step, Identifying metal layer, spacing and width for the ring, stripes and rail, connecting them with vias, check if the tracks are perfectly on the pins of the design, check the drcs and check if there are floating points.

4.1.2 power planning commands:

- remove_pg_via_master_rules -all
- remove_pg_patterns -all
- Removes all predefined power ground
- remove_pg_strategies -all
- remove_pg_strategy_via_rules -all

Removes all predefined power ground via rules, patterns, strategies and strategy_via_rules.

- set_pg_via_master_rule PG_VIA_3x1 -cut_spacing 0.25 -via_array_dimension {3 1}

Creates a new via master rule for the power and ground network.

-cut_spacing option: Specifies the values between cuts in a simple array via.

-via_array_dimension option: Specifies the via array dimension size by integer values for column and row.

- create_pg_ring_pattern ring \

  -horizontal_layer M8 -vertical_layer M9 \

  -horizontal_width 5 -vertical_width 5 \

  -horizontal_spacing 2 -vertical_spacing 2

This command creates a power ground (PG) ring pattern. The ring pattern specifies the horizontal and vertical ring width values, layer names and spacing values options.

- create_pg_std_cell_conn_pattern rail_pattern -layers {M1} -rail_width {0.094 0.094}

This command creates a standard cell rail connection pattern and specifies the layer and rail's width.

- create_pg_mesh_pattern m8_mesh -layers {{{horizontal_layer: M8} {width: 1} {spacing: 9} {pitch: 20} {offset:5}}}
- create_pg_mesh_pattern m7_mesh -layers {{{vertical_layer: M7} {width: 1} {spacing: 9} {pitch: 20} {offset:5}}}

Creates a power ground mesh pattern. The mesh pattern specifies the horizontal and vertical mesh width values, layer names, spacing values, pitch values and offset values options.

- set_pg_strategy ring_s -pg_regions {top_power_ring_region} \

  -pattern {{name: ring} {nets: "VSS VDD" }} \

  -extension { {{stop:design_boundary}} }

- set_pg_strategy rail_strat -pattern {{pattern: rail_pattern} {nets: VDD VSS}} -core
- set_pg_strategy m7_s -core -extension {{stop:design_boundary_and_generate_pin}} -pattern {{name: m7_mesh} {nets: VDD VSS}}
- set_pg_strategy m8_s -core -extension {{stop:design_boundary_and_generate_pin}} -pattern {{name: m8_mesh} {nets: VDD VSS}}

This command specifies the power ground network creation strategy.

-pattern option: specifies the power ground pattern to be instantiated in this strategy (The name keyword followed by pattern_name specifies the name of the pattern) and (The nets keyword followed by real_net_list specifies the list of net names to be mapped into the symbolic list of nets in pattern_name.).

-core option: Uses the core area as the power ground region.

-extension option: specifies how a group of power and ground nets should extend beyond the power grid. Design_boundary specifies the boundary of the current design. Design_boundary_and_generate_pin specifies extension to the design boundary and generating pins. Core_boundary specifies the core area boundary.

- compile_pg -strategies ring_s
- compile_pg -strategies rail_strat
- compile_pg -strategies m8_s
- compile_pg -strategies m7_s

This command creates a power and ground network including straps, rings, macro connections, standard cell connections and via connections based on the specified power ground strategies.

- create_pg_vias -to_layers M7 -from_layers M1 -via_masters PG_VIA_3x1 -nets VDD

- create_pg_vias -to_layers M7 -from_layers M1 -via_masters PG_VIA_3x1 -nets VSS

Creates PG vias in the specified bounding box for the specified PG nets.

5.1placement

5.1.1overview:

In ASIC (Application-Specific Integrated Circuit) design flow, "placement" refers to the crucial step where physical locations for various logic gates and components on the chip are determined. This process significantly impacts the performance, power consumption, and overall feasibility of the integrated circuit, Placement involves positioning standard cells within the defined floorplan, considering factors such as congestion, cell density, and overall accessibility for routing, The output of this stage is a preliminary placement layout, where components are positioned on the chip surface.

Inputs:

- Netlist of gates and wires.
- Floorplan , Power plan and Technology constraints.

Output:

- All cells legally located in the floorplan.

The goal of placement is to optimize several key parameters:

1. Timing: Ensuring that signals propagate through the circuit within specified timing constraints. This involves minimizing delays and ensuring that critical paths meet timing requirements.

2. Area: Efficiently utilizing the available silicon real estate. Compact placement reduces the overall chip size, which in turn lowers production costs and can improve signal integrity by reducing wire lengths.

3. Power: Minimizing power consumption by strategically placing components to reduce switching activities and optimizing power distribution networks.

4. Signal Integrity: Maintaining the quality of signals throughout the chip by minimizing noise, crosstalk, and ensuring proper routing resources.

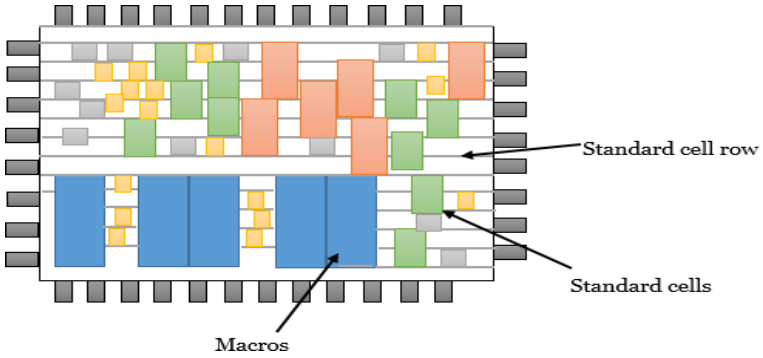

Figure 8 placement

Advanced placement methodologies take into account the hierarchical nature of ASIC designs, where large designs are divided into smaller blocks for manageable optimization. This hierarchical approach allows for scalability and facilitates easier design closure.

The placement process is iterative, where initial placement is refined through multiple optimization steps to achieve desired objectives. Designers also leverage floorplanning techniques to define high-level placement constraints early in the design phase, which helps guide the placement process towards meeting overall design goals.

5.1.2 Coarse (Global) Placement vs Detailed (Legalized) Placement:

Global placement and detail placement are two sequential stages within the ASIC design flow that collectively determine the physical layout of the integrated circuit.

Global Placement: This initial stage focuses on placing logical components of the design onto the chip's physical layout while adhering to high-level constraints and objectives. Algorithms used in global placement aim to achieve a rough approximation of optimal placement by considering factors such as wirelength minimization, timing requirements, and overall chip area utilization. Global placement lays the foundation for subsequent detailed

placement by providing an initial arrangement of components that meet basic design goals.

Detail Placement: Following global placement, detail placement refines the initial placement by considering more detailed physical and timing requirements. This stage involves precise placement of individual logic gates and components, taking into account local routing congestion, signal integrity, and manufacturability constraints. Detail placement algorithms iteratively optimize the placement to improve timing closure, reduce wirelengths, and ensure adherence to design rules specified by the foundry process. The goal is to achieve a final placement solution that meets all design criteria before proceeding to routing and manufacturing stages.[4]

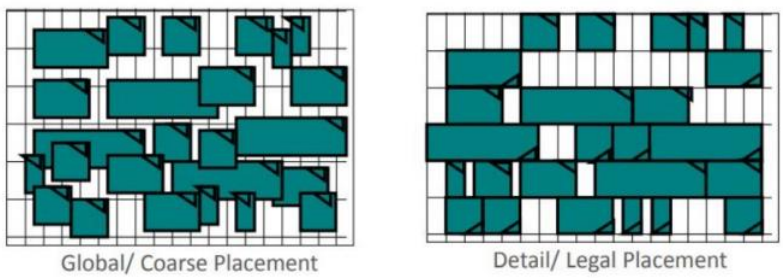

Figure 9 Coarse vs Detail Placement

5.1.3 Placement optimization:

Placement optimizations are crucial for achieving optimal timing closure and addressing high fanout nets. Various strategies include:

- Optimizing Timing: Reducing wirelength, dividing long nets using buffers, layer promotion, resizing gates to meet timing.
- High Fanout Nets Fixing: Addressing nets with high fanout through tie-cell insertion and scanchain reordering.
- Logical Restructuring: Gate composition or decomposition to enhance worst negative slack (WNS) or total negative slack (TNS).[4]

5.1.4 Tie-Cell Insertion

Tie-cells are inserted to connect the input of logic gates that need to be linked to VDD or VSS. This prevents damage to the gate oxide under the poly gate due to power noise.

5.1.5 Scan-Chain Reordering

Scan-chain reordering optimizes the arrangement of scan chains to improve routing. This process ensures easier routing of scan chains and reduces congestion.[2]

5.1.6 Logical Restructuring

Logical restructuring involves gate composition or decomposition to enhance worst slack or total negative slack, improving overall placement quality.[3]

5.1.7 Congestion in Placement

Congestion occurs when the number of required routing tracks exceeds the available tracks. It can lead to routing difficulties and must be minimized or eliminated. Congestion maps are useful for evaluating total congestion and identifying hotspots.[3]

Reasons for Congestion:

- High standard cell density in a small area.
- Placement of standard cells near macros.
- High pin density at the edge of the macro.
- Macro pins near the core area boundary.
- Blind double spacing/width for clock tree synthesis (CTS) in lower metal layers near pins.

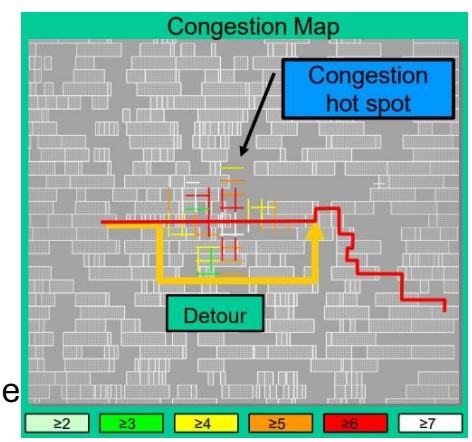

Figure 10 Global congestion map

5.1.8 Placement Constraints

Placement constraints, including blockages, bounds, and keep-out margins, are used to reduce or avoid congestion and enhance timing.

5.1.9 Strategies to Fix Congestion

- Modify the floorplan to mark areas for low utilization.
- Align bus signal pins.
- Increase spacing between macros.
- Add blockages and halos.
- Adjust core aspect ratio and size.
- Optimize power grid.

5.1.10 Commands used for placement:

Open powerplanned block

open_block -edit $DESIGN_NAME:${DESIGN_NAME}_powerplanned :

Opens an existing saved block for editing or viewing and sets that block as the current block.

link :

Resolves block references , The command determines the initial block view to use to resolve a reference based on the view switch list for the current block.

source /home/ICer/GP/PULPino/1_syn/scripts/dont_use_cells.tcl :

Opens a file that contain cells that causes violations and DRCs to make the tool don't use.

Pre Place Checks

remove_corners estimated_corner :

Removes one or more corners in multi corner analysis, as this command removes the specified corners from memory.

set_scenario_status func_slow -hold false :

set_scenario_status -setup false func_fast :

ConFigures analysis settings for scenarios, This command enables or disables timing analysis, power analysis, and DRC checking for scenarios in the current design. Setup analysis, hold analysis, power analysis, and various types of DRC and EM checking can be conFigured independently.

Note that the create_scenario command creates a scenario and makes it active for all analysis types except cell and signal EM. The set_scenario_status command will generally be needed after a scenario is created to set only the desired analysis types.

-setup true | false : Specifies whether setup analysis is turned on for the specified scenarios.

-hold true | false : Specifies whether hold analysis is turned on for the specified scenarios.

check_design -checks pre_placement_stage : checks on pre placement stage

check_design -checks physical_constraints : checks on physical constraints

Runs pre-defined or user-defined checks on current design. Messages generated by the checks run, are captured into an EMS database file and also into a text log file. Summary of the messages generated is also reported.

analyze_lib_cell_placement -lib_cells * :

This command tests a collection of library cells for their ability to be placed in a the current design. For each cell in the collection, trials free sites are selected at random. If a region is specified then the list of random sites is limited to free sites within that area. If the cell can be placed at that site in any orientation, then it is considered to pass at that site. If it cannot, it is considered to fail at that site. The pass rate for the cell is the number of passing sites divided by the number of sites tested.

After all of the cells have been analyzed, a report is produced listing cells in order of increasing pass rate. The length of this report is limited by the max_cells argument. It is also limited by the threshold argument. No cells whose pass arate is above this will be reported.

-lib_cells : Specifies the library cells to analyze for placeability. This is a required option.

Placement Begins and app_options

remove_ideal_network -all :

This command removes the ideal network property from a set of ports or pins in the design that were previously marked as ideal sources using the set_ideal_network command. You specify only the source of the network. The ideal network is automatically updated.

place_opt :

Place and optimize the current design, as The default behavior of this command:

(1) places the current design

(2) optimizes the placed design for timing, electrical DRC violations, area, power, and routability

(3) performs incremental placement to optimize timing and routability

(4) further optimizes the placed design

(5) legalizes the design placement at the end.

legalize_placement :

Executes detailed placement on the current design, as This command performs placement on the current design after coarse placement has already been performed.

When you run this command, the design should contain a floorplan with a site array specified and all cells and blockages in the design should have an initial location.

app options : Application options are set using the set_app_options command, and they may be specified globally or with respect to a design, and they are used to make the tool more applicable for specific issues.

set_app_options: This command is used to set application-specific options within the ASIC design tool environment.

set_app_options -name place.coarse.continue_on_missing_scandef -value true :

this command conFigures the ASIC design tool to handle missing scan definitions during the coarse placement phase, ensuring that the design process can proceed smoothly and adaptably in early stages of development.

set_app_options -name time.disable_recovery_removal_checks -value false :

When true, disables recovery and removal timing analysis. When false (the default), the tool performs recovery and removal checks.

In the context of placement in ASIC design, recovery and removal checks are important steps that ensure the physical layout of the integrated circuit (IC) meets specific timing requirements and design constraints:

Recovery Checks : ensure that the timing requirements between sequential elements (like flip-flops) in the circuit are met after placement and routing.

Removal Checks : ensure that no unnecessary components, such as redundant buffers or logic gates, are introduced during placement that could degrade timing performance or increase power consumption.

Both recovery and removal checks are integral to the iterative optimization process during placement. They ensure that the physical implementation of the design meets timing specifications, minimizes area utilization, and maximizes the efficiency of the ASIC layout before moving on to subsequent stages like routing and verification.

set_app_options -name time.disable_case_analysis -value false :

it ensures that the ASIC design tool includes case-specific timing analysis in its overall timing closure process, contributing to a thorough verification of timing constraints in the designed integrated circuit, when false (the default), constant propagation is performed in the design from pins either that are tied to a logic constant value, or for which a set_case_analysis command is specified. For example, a typical design has several pins set to a constant logic

value. By default, this constant value propagates through the logic to which it connects. When the app option time.disable_case_analysis is true, case analysis and constant propagation are not performed.

set_app_options -name opt.common.user_instance_name_prefix -value place :

Users can specify a string to be prepended to new cell names in optimization by setting app option value of opt.common.user_instance_name_prefix. This applies to cell names created during data optimization. Names created during clock optimization are controlled by cts.common.user_instance_name_prefix , conFigures the ASIC design tool to apply the prefix place to user-defined instance names, enhancing organization and manageability of custom modules or cells within the design flow.

For example, if a cell is expected to be created by place_opt and to have name as buft_p_60, then by setting opt.common.user_instance_name_prefix as "place_" before place_opt, the cell's name would be place_buft_p_60 after place_opt.

set_app_options -name opt.timing.effort -value high :

Enable higher effort timing recovery in preroute flows, and used to prioritize aggressive timing optimization efforts, aiming to achieve optimal timing closure and maximize the performance of the integrated circuit design.

set_app_options -name opt.area.effort -value ultra :

Enable higher effort area recovery in preroute flows, optimize chip area, aiming to achieve the smallest possible physical footprint for the integrated circuit design.

set_app_options -list {place.legalize.legalizer_search_and_repair {true}} :

Enable search and repair functionality within the legalization process during placement, ensuring that the physical layout of the design adheres to all necessary design rules and constraints.

Legalization : In ASIC design, legalization refers to the process of adjusting or modifying the physical placement of cells (logic gates, memory elements, etc.)

to ensure they meet design rules and constraints imposed by the manufacturing process.

Search and Repair : The term "legalizer search and repair" suggests that the tool will perform a search to identify placement violations or inconsistencies and then apply repairs or adjustments to bring the placement back into compliance with design rules.

6.1 Clock Tree Synthesis

6.1.1 Overview

Clock Tree Synthesis (CTS) is the process of designing and implementing a clock distribution network throughout an IC design. Its primary goal is to deliver a clean, low-skew clock signal to all sequential elements (like flip-flops) within the chip. This ensures synchronous operation and timing integrity across the entire design.[4]

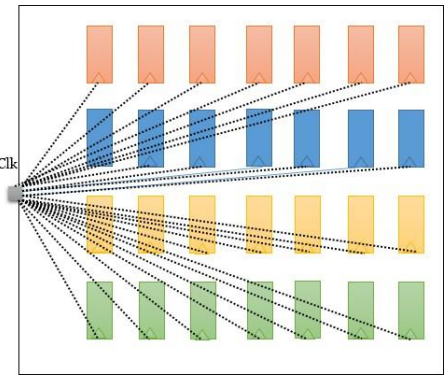

Figure 11 CTS

6.1.2 Clock Parameters

Skew Definition: Difference in clock arrival time at two spatially distinct points.

Jitter Definition: Difference in clock period between different cycles.

Insertion Delay Definition: Delay from the clock source until registers.

Capture Clock Edge: The edge of the clock for which data is detected.

Launch Clock Edge: The edge of the clock where data is launched.

Local Skew: Difference in the arrival of the clock signal at the clock pin of related flops.

Global Skew: Difference in the arrival of the clock signal at the clock pin of non-related flops.

Positive Skew: Capture clock comes late compared to launch clock, leading to potential hold violations.

Negative Skew: Capture clock comes early compared to launch clock, potentially causing setup violations.

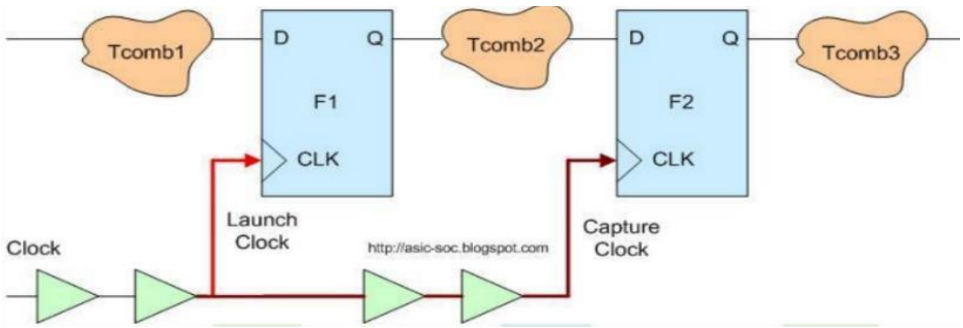

Figure 12 CTS parameters

6.1.3 Source Delay, Network Delay, and Uncertainty:

Source Delay (Source Latency): Delay from the clock origin point to the clock definition point.

Network Delay (Insertion Delay): Delay from the clock definition point to the clock pin of the register.

Uncertainty: Time difference between arrivals of clock signals at registers in one clock domain or between domains.[4]

6.1.4 CTS goals

CTS aims to build a buffer/inverter network to balance relative delays of Flip-Flops (FFs) within a clock domain.

Goals include minimizing global skew, optimizing power, area, and ensuring signal integrity.[5]

6.1.5 Clock Tree Constraints :

Clock tree constraints include symmetric buffers/inverters, routing layers, NDR rules, target skew, max transition, max capacitance, and CTS cell spacing.

6.1.6 Clock Tree Synthesis Execution :

CTS execution involves commands like clock_opt and set_clock_tree_exceptions to define stop, float, or exclude pins.

6.1.7 Effects of CTS :

CTS introduces clock buffers, potentially increasing congestion.

Non-clock cells may be relocated, leading to new timing and max transition/capacitance violations.

6.1.8 Clock Tree Optimization :

Clock tree optimization involves sizing, relocation, buffer insertion, and gate sizing to achieve useful skew.[6]

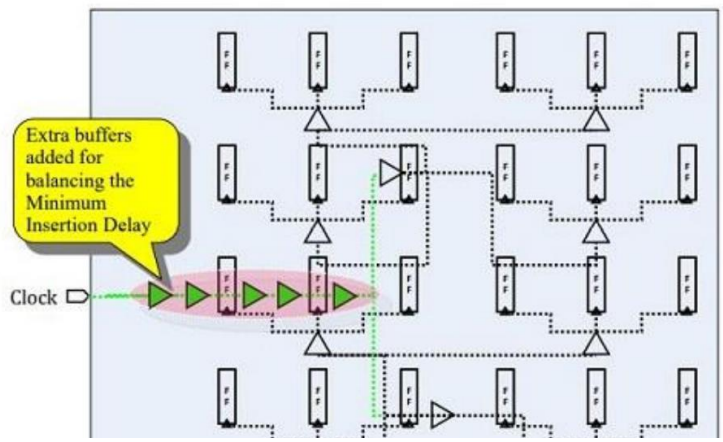

Figure 13 Buffer insertion

6.1.9 Commands used for placement

Open powerplanned block

open_block -edit $DESIGN_NAME:${DESIGN_NAME}_placed :

Opens an existing saved block for editing or viewing and sets that block as the current block.

link :

Resolves block references , The command determines the initial block view to use to resolve a reference based on the view switch list for the current block.

Placement Begins and app_options

set_dont_touch_network :

This command sets the clocks, pins, or ports in the current design as source of network dont_touch. The tool will assign dont_touch attributes to all cells and nets in the transitive fanout of these objects so that they are not modified or replaced during optimization. A dont_touch attribute assigned by the tool using the set_dont_touch_network command overrides the false dont_touch attribute value from the set_dont_touch command.

set_dont_use [get_lib_cells * *] :

This command excludes specific library cells matching the pattern between (* *) from being used in the ASIC design flow, providing flexibility and control over the design's implementation and optimization.

set_lib_cell_purpose -include cts * * :

This command conFigures the tool to include library cells matching what is between (**) for use in clock tree synthesis, ensuring efficient and effective distribution of clock signals across the integrated circuit design.

Checkers Pre CTS

check_design -checks pre_clock_tree_stage : checks on the design pre CTS

Runs pre-defined or user-defined checks on current design. Messages generated by the checks run, are captured into an EMS database file and also into a text log file. Summary of the messages generated is also reported.

set_propagated_clock [get_clocks * *] :

This command specifies that the clock signal (or clocks associated with clock signal) should be treated as a propagated clock for timing analysis purposes, ensuring robust timing closure and synchronization in the integrated circuit design.

remove_ignored_layers -all :

Removes all RC congestion ignored layers between design minimum and maximum routing layer inclusive.

set_ignored_layers -min_routing_layer M* * -max_routing_layer M* * :

It conFigures the tool to exclude routing paths on metal layers from minimum routing layer to maximum routing layer during the routing phase, allowing for customization and optimization of the routing process based on design and manufacturing considerations.

create_routing_rule ROUTE_RULES_1 -multiplier_spacing 2 -multiplier_width 2 :

This command defines non-default routing rules in the design,

-multiplier_width width_multiplier : Specifies the multiplier to apply to the layer widths in this routing rule.

-multiplier_spacing spacing_multiplier : Specifies the multiplier to apply to the layer spacings in this routing rule.

set_clock_routing_rules -rules ROUTE_RULES_1 -min_routing_layer M* * -max_routing_layer M* * :

It conFigures the tool to enforce routing rules (ROUTE_RULES_1) specifically for clock signals on routing layers from minimum layer to maximum layer, ensuring proper timing and synchronization throughout the integrated circuit design.

set_clock_tree_options -target_latency 0.000 -target_skew 0.000 :

This helps in achieving timing closure by ensuring that clock signals propagate with minimal delay and skew, meeting stringent timing requirements, as it conFigures the tool to optimize the clock distribution network (clock tree) by setting stringent targets for latency and skew, aiming for minimal delay and perfectly synchronized clock signals throughout the integrated circuit design.

set cts_enable_drc_fixing_on_data true :

It makes the tool to automatically fix DRC violations on data paths during the Clock Tree Synthesis (CTS) stage, improving efficiency and ensuring compliance with design rules in the integrated circuit design process.

clock_opt :

Synthesize and route the clocks in the current design and then further optimize the design based on the propagated clock latencies.

The default behavior of this command is as follows:

(1) Synthesizes and optimizes the clock trees.

(2) Completes the detailed routing of the clock trees.

(3) Further optimizes the design for timing,electrical DRC violations, area, power, and routability, based on actual propagated clock latencies, and legalizes the design placement at the end.

app options : Application options are set using the set_app_options command, and they may be specified globally or with respect to a design, and they are used to make the tool more applicable for specific issues.

set_app_options: This command is used to set application-specific options within the ASIC design tool environment.

set_app_options \-name cts.common.user_instance_name_prefix -value "CTS_" :

This app option sets a prefix for new cell names created by clock tree synthesis tools. The default value of the prefix is a NULL string.

set_app_options -name clock_opt.hold.effort -value high :

Enable higher effort hold fixing in clock_opt flows. The hold fixing effort can be: none, low, medium, or high. Setting to none disables hold fixing.

set_app_options -name opt.timing.effort -value high :

Control the optimization effort for timing and the effort is set to be high.

set_app_option -name opt.power.mode -value total :

Enable dynamic power optimization in preroute flows.

There are three modes for dynamic power optimization: none, dynamic and total.

When this option is set to none, dynamic power optimization will be turned off.

When this option is set to dynamic, only switching and internal power will be optimized.

When this option is set to total, the total power consumption comprising leakage, switching and internal power will be optimized.

set_app_option -name opt.power.effort -value medium :

Control the optimization effort for power.

7.1 Routing

7.1.1 Routing overview

routing is the process of establishing the interconnections or wiring between the placed components. It involves determining the paths of the wires or metal tracks that connect the pins of the components according to the logical functionality of the circuit. It considers design constraints (e.g., Wirelength, signal congestion, and signal integrity)

Its algorithms utilize various techniques to optimize the process such as global routing and detailed routing, hey aim to find the most efficient and manufacturable wire routes while adhering to design rules and constraints. The final output of the routing stage is a complete routing solution that satisfies the design requirements.

In grid-based routing system each metal layer has its own tracks and preferred routing direction which are defined in a unified cell in the standard cell library.

There are four steps of routing operations:

- Global routing
- Track assignment
- Detail routing
- Search and repair

Global Route: assigns nets to specific metal layers and global routing cells. Global route tries to avoid congested global cells while minimizing detours.

Global route also avoids pre-routed P/G, placement blockages and routing blockages.

Track Assignment (TA): assigns each net to a specific track and actual metal traces are laid down by it. It tries to make long, straight traces to avoid the number of vias. DRC is not followed in TA stage. TA operates on the entire design at once.

Detail Routing: tries to fix all DRC violations after track assignment using a fixed size small area known as "SBox". Detail route traverses the whole design box by box until entire routing pass is complete.

Search and Repair: fixes remaining DRC violations through multiple iterative loops using progressively larger SBox sizes.

Inputs of Routing

- Netlist
- All cells & ports should be legally placed with clock tree structure & CTS DEF file
- NDRs
- Routing blockages
- Technology data (metal layers (lef, tech file etc.), DRC rules, via creation rules, grid rules (metal pitch) etc...)

7.1.2 Routing main commands in PnR flow:

check_physical_design –stage pre_route_opt

This command checks the readiness of the current design for IC Compiler with:

-stage option specifies which stage to check. The checks performed by the **check_physical_design** command depend on the specified stage.

check_routeability

Verifies that the current design is routable and Check's pin access points, cell instance wire tracks, pin out of boundaries, min-grid and pin design rules and

blockages to ensure they meet the design requirements. It performs a check of the design for optimization in order to substantiate any errors in the design that might need to be fixed or what could help to improve the design. This must currently be run on a placed design.

remove_ignored_layers -all

Removes ignored routing layers in congestion analysis and RC estimation. This command can also remove the design minimum and maximum routing layers if those layers have been already set.

set_ignored_layers -min_routing_layer $MIN_ROUTING_LAYER -max_routing_layer $MAX_ROUTING_LAYER

This command sets the routing layers that are ignored during congestion analysis and RC estimation. You can use this command when you do not want to route nets on certain routing layers. If you set the minimum and maximum routing layers for a design by using this command, the router respects these settings. If you set the minimum routing layer for the design to a certain layer, this command automatically sets all the routing layers below this layer as fully blocked layers. The same behavior also applies to the maximum routing layer setting.

route_auto

Performs global routing, track assignment, detail routing, and search and repair in one step

route_opt

This command optimizes the current routed design for timing, electrical DRC violations, area, and power. It performs legalization and ECO routing on the optimized design. That marks the end of the PNR stage of the design, LVS

checks are done and a map of congestion and cell density are generated to view the resulted NDM along with timing, constrains and QOR reports .

8.1 Chip Finishing

8.1.1 Chip finishing overview

To help address some of the process design issues during chip manufacturing, IC provides some commands that provide options which can be applied.

Chip _finishing consists of four basic steps:

- Metal layers spreading and widening
- Standard cell _filling
- Redundant vias insertion
- Metal _filling

Metal layers spreading and widening is performed to protect minimum-width interconnections from opens and minimum-spacing interconnections from shorts, this could happen because of any random particles that could fall on the chip and damage it during fabrication process. But spreading and widening may introduce new timing violations, to avoid such a problem we may prevent spreading and widening from timing-critical nets by setting setup threshold and hold threshold. These thresholds mean if any timing path has a setup margin or hold margin below or equal to these values, no spreading or widening is performed on any net of these paths.

Standard cell _filling is performed in the empty locations in the standard cell rows to make the chip uniform in density and to improve the yield of the chip.

Redundant vias insertion is an insertion of extra vias beside the original vias because if any via of both original via an extra via is defective, connection

between metal layers will not fail as another contact will still connect metal layers together.

Metal _filling is performed to protect metal interconnections in low metal density regions from over-etching during fabrication process. Timing driven metal _filling specially is performed to preserve timing on critical nets, metal fill near critical nets on the same layer, upper layer, and lower layer are removed or trimmed.

8.1.2 Finishing main commands:

A typical flow is:

1. Use create_stdcell_fillers to insert DECAPs
2. Update PG connections by using connect_pg_net -automatic
3. Use remove_stdcell_fillers_with_violation to remove DECAPs with violations
4. Use create_stdcell_fillers to insert non-metal fillers
5. update PG connections by using connect_pg_net -automatic

After filler insertion, use the check_legality command to verify filler placement legality.

```
create_stdcell_filler -lib_cell $pnr_DCAP_fillers
```

This command fills empty spaces in standard cell rows with instances of reference filler cells from the library. You should perform placement (and clock tree synthesis, if applicable) before adding filler cells.

```
connect_pg_net -automatic
```

This command creates the power and ground network for your design by connecting the power, ground, and tie_off pins to the power and ground nets in your design.

```
remove_stdcell_fillers_with_violation
```

This command deletes the specified filler cells that have routing DRC violations.

8.1.3 Reports and checks:

Finally, as we output out PNR into the design library, we run a few check commands to get an idea of how the violations look like:

check_lvs

Verify_lvs is an important command that checks for the inconsistencies in the physical layout of the deisign, such as the number of shorted or open wires, this command was used in the previous stages as well, in order to keep track of how each of the commands affect the violations.

report_timing –delay_type max
report_timing –delay_type min

Report_timing is another command that was also in almost every stage of the PNR, as it returns the setup and hold slack on which we base our script strategy on.

report_qor

reports Quality of Results information about the design. This includes timing information, cell count details, and statistics such as combinational, noncombinational, and total area. Timing-related information is reported separately for each scenario specified by the -mode, -corner, and -scenario options.

write_parasitics –output {./***output***/ DESIGN_NAME.***spef***}

writes parasitics for the current design to SPEF files.

write_verilog ../output/${DESIGN_NAME}_finish.v

writes a hierarchical Verilog netlist from the current design.

write_gds This command writes out the physical data of a top-level design as a GDSII stream file for tape out or for export to third-party tools.

-layer_map $GDS_MAP_FILE Specifies a layer mapping file that customizes the translation of layer, data type, and purpose between the tool's data mode and the GDSII file.

-keep_data_type \

-fill include Writes all geometries, including fill data

-output_pin all \ Specifies the manner in which pin objects are written into the GDSII file. -all Writes both the text and geometry of each pin.

-lib_cell_view frame \ Specifies the name of the view used for writing leaf-level reference library cells to the GDSII file.

-long_names \ write long hierarchical names without modification. By default, the command limits each name string to 32 characters

9.1 PRIME TIME

9.1.1 STA Overview

In the world of modern Very-Large-Scale Integration (VLSI) design, timing analysis is crucial to ensure that integrated circuits (ICs) meet performance specifications. Static Timing Analysis (STA) is a method used to validate the timing performance of a digital design by checking its timing constraints without requiring dynamic simulation. It focuses on the propagation of signals through the combinational paths between sequential elements, ensuring that signals meet setup and hold time requirements across all possible timing paths. STA is indispensable in ensuring that a design operates correctly at its intended clock frequency under worst-case conditions.

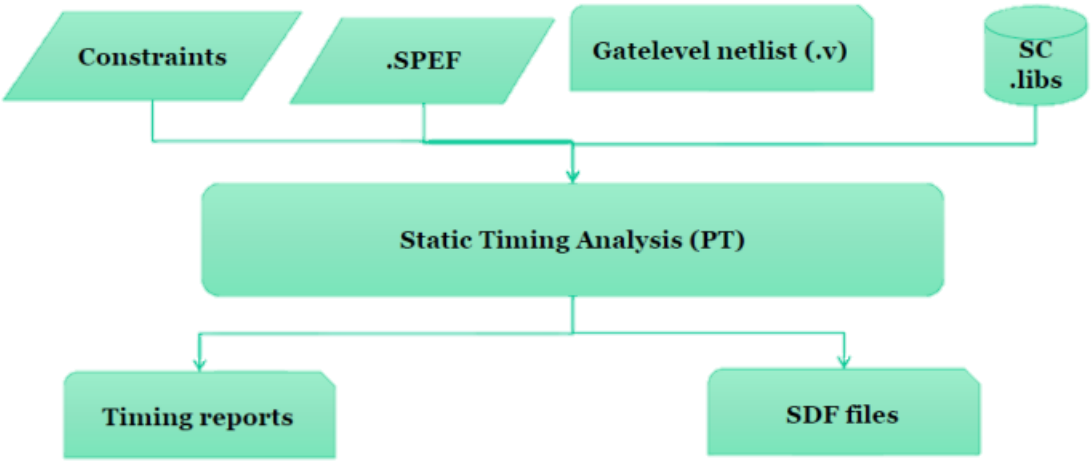

Figure 14 STA flow using primetime

9.1.2 PrimeTime Overview

is a leading STA tool widely used in the semiconductor industry to perform accurate and comprehensive timing analysis. Used for timing closure and signoff. It provides detailed insights into the timing performance of digital designs, enabling engineers to identify and rectify timing violations early in the design process.

9.1.3 Key Features of PrimeTime

9.1.3.1 **Verify timing**

- Verify timing between synchronous clocks
- Does not verify functionality

9.1.3.2 **Is Exhaustive**

- use formal ,mathematical techniques instead of vectors.
- does not use dynamic logic simulation.

9.1.3.3 **Is Fast**

- significantly faster than gate level netlist
- orders of magnitude faster than transistor level simulation.

9.1.4 Incremental Timing Analysis

PrimeTime offers incremental analysis capabilities, allowing designers to re-analyze only the parts of the design that have changed, significantly speeding up the timing closure process.

9.1.5 ECO (Engineering Change Order) Support

PrimeTime facilitates ECO changes, allowing designers to make late-stage modifications to the design for fixing timing and DRCs issues without requiring a full redesign.

9.1.6 Prime Time in the implementation flow

- Run at the gate level netlist after design transformation
    - before manufacturing
    - sometimes between transformation within tools
- Run whenever there is a change

- in design netlist
- in design constraints

9.1.7 Prime time inputs and outputs

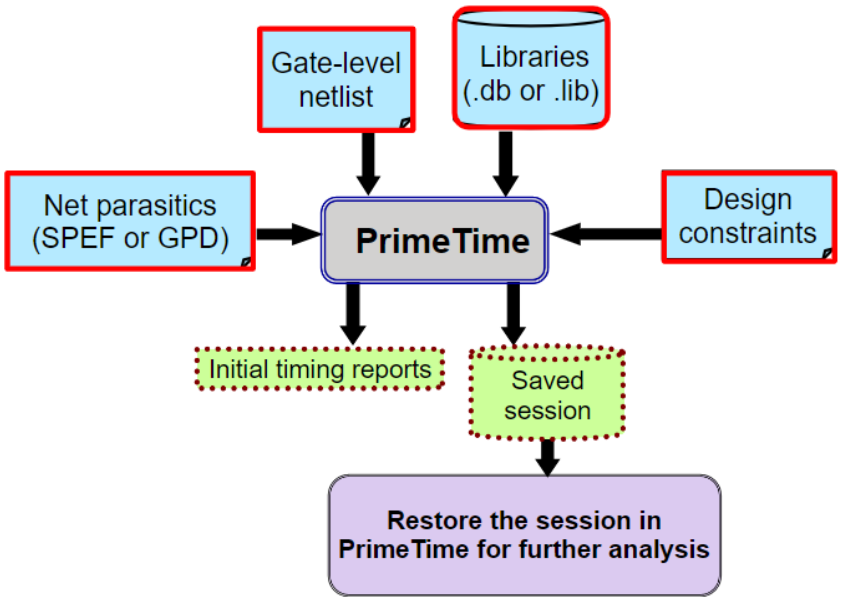

Figure 15 PrimeTime inputs and outputs

9.1.8 Timing Analysis Flow in Prime Time

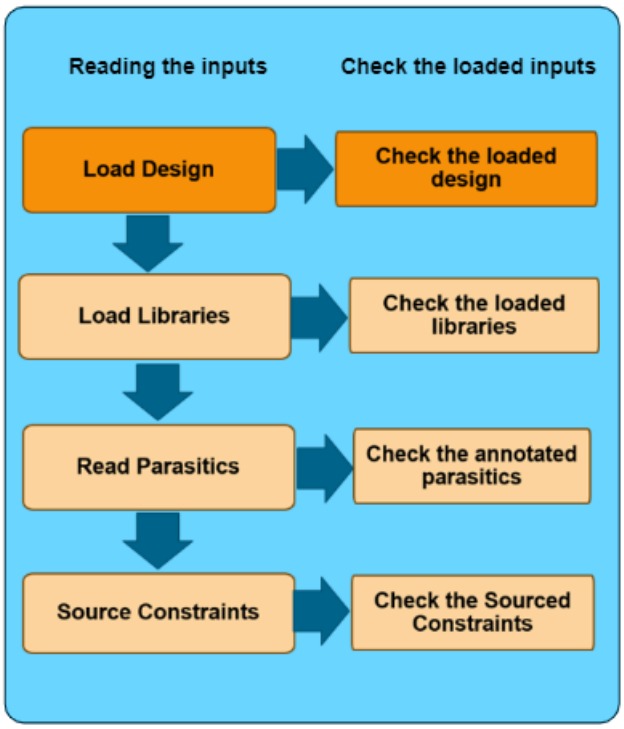

Figure 16 Timing Analysis Flow in PrimeTime

9.1.9 Load design and check

-first, read the gate level netlist :
read_verilog $NETLIST_FILES
-then the current design is set for the top level design :
current_design $DESIGN_NAME
-we can check what is the current design

check_design
-we can check what designs are loaded by command
get_designs *

9.1.10 Load Libraries and check

-Set search Path where u can set libraries foe different design files so you don't need to specify them each time you are loading them to the design

set search_path ". $ADDITIONAL_SEARCH_PATH $search_path"
-set variable link path : will specify the name of the DBs for the standard cells or macros that instantiated in the netlist

```
set link_path "* $target_library $ADDITIONAL_LINK_LIB_FILES"
```

-then u will link the design : will load the libraries and resolve references so it will link the instance in the netlist of the library you have specified in the link path variable

```
link_design -verbose
```

-now to check if the libraries are loaded you can use this command

```
list_libs
```

9.1.11 Read parasitic

We can either use spef or GPD format

```
read_parasitics  $PARASITIC_FILES
then we can check perastic annotation
report_annotated_parasitics -check > $REPORTS_DIR/rap.report
```

9.1.12 source constraints and check for corners

-we can also read SDC or TCL format

```
source $cons_file
```

-to check if the constraints are complete we can use

```
check_timing -verbose
```

```
-to exmine clock and ports constraints
```

```
report_clock -skew -attributes
```

```
report_port -verbose
```

-to exmine design constrains

```
report_design
```

to report different design information such as wire load model or voltage of the design

```
report_exceptions -ignored
```

to report exception in the design such as multi cycle

9.1.13 confirm constrints completeness

```
check_timing -verbose
```

will check for different types of timing checks such as no_input_delay or no_driving_cell

```
pt_shell> check_timing -verbose
Information: Checking 'no_input_delay'.
Information: Checking 'no_driving_cell'.
Information: Checking 'unconstrained_endpoints'.
Information: Checking 'unexpandable_clocks'.
Information: Checking 'latch_fanout'.
Information: Checking 'no_clock'.
Information: Checking 'partial_input_delay'.
Information: Checking 'generic'.
Information: Checking 'loops'.
Information: Checking 'generated_clocks'.
Information: Checking 'pulse_clock_non_pulse_clock_merge'.
Information: Checking 'pll_configuration'.
check_timing succeeded.
1
```

Figure18:check Timing command

9.1.14 Update Timing

Invoke and explicit update timing once

```
update_timing -full
```

Check update timing results by generating summary reports such as

```
report_global_timing
report_qor
report_constraint -all_violators
report_analysis_coverage
report_global_slack
```

saving_session and exit

```
save_session my_savesession
exit
```

We used this command to save the current PrimeTime session to make it easy to do many
iterations and show the difference between them.

A Saved session contains the following information for restore:

- Link design and loaded library

- Clocks, timing exception and other constraints
- Operating conditions
- Back-annotated SDF delays and parasitic
- Variable setting
- Netlist edits (insert_buffer , size_cell , swap_cell)
- Analysis data
- Cross_coupled delay and noise data

9.2 ECO(ENGINEERING CHANGE ORDER)

9.2.1 Overview of ECO in VLSI Design

In the design and verification cycle of VLSI, Engineering Change Orders (ECOs) are essential for making last-minute design modifications without the need for a complete redesign. ECOs are typically applied to fix issues such as timing violations, design rule checks (DRCs), and functional bugs identified late in the design process or during signoff. These changes are crucial to meet the final design specifications and ensure the manufacturability and performance of the IC.

PrimeTime supports ECOs by offering functionalities like `fix_eco_timing` and `fix_eco_drc`, which streamline the process of implementing and verifying these changes.

9.2.2 Fixing Timing Violations

The `fix_eco_timing` command in PrimeTime automates the process of correcting timing violations in the design. This command can make various modifications to the netlist to ensure that timing constraints are met without adversely affecting the design.

fix_eco_timing with –type setup or hold

Setup fixing supports either size_cell or insert_buffer – Hold fixing supports any (and all) of size_cell or insert_buffer or insert_buffer_at_load_pins

9.2.3 Addressing DRC Violations

The `fix_eco_drc` command in PrimeTime targets the resolution of Design Rule Check (DRC) violations. These violations can arise due to constraints

related to the physical layout or electrical properties that must be met to ensure reliable fabrication and operation.

Use fix_eco_drc with –type max_transition or noise or max_fanout or max_capacitance, and specify physical mode as either open_site or occupied_site

–Max transition and noise fixing support both size_cell and insert_buffer

9.2.4 Exporting ECO Changes

Once timing and DRC violations are fixed using `fix_eco_timing` and `fix_eco_drc` in PrimeTime, the modifications need to be reflected in the physical design tool, such as ICC2. The `write_changes` command in PrimeTime facilitates this by generating a change file containing the necessary modifications.

```
write_changes -format icc2tcl -output ECO.tcl
```

9.2.5 Applying ECO Changes in ICC2

The change file generated by PrimeTime needs to be applied to the physical design in ICC2. This process involves several key steps: placing ECO cells, routing ECO changes, and connecting power and ground nets.

Key Commands for Applying ECO Changes in ICC2:

9.2.6 Place ECO Cells:

Use `place_eco_cells` to place the cells modified by ECO in a way that minimizes physical impact.

```
place_eco_cells -legalize_mode minimum_physical_impact \
-eco_changed_cells -legalize_only -displacement_threshold 10
```

`-legalize_mode minimum_physical_impact` ensures that the placement has minimal disruption to the existing layout.

`-eco_changed_cells` targets only the cells affected by the ECO.

`-legalize_only` restricts the command to only legalize placement without additional optimizations.

`-displacement_threshold 10` sets the maximum allowable displacement for cells being legalized.

9.2.7 Route ECO Changes:

Use `route_eco` to update the routing for the modified cells.

```
route_eco
```

9.2.8 Connect Power and Ground Nets:

Ensure that the power and ground connections are properly updated for the modified cells using

```
connect_pg_net -automatic
```

9.2.9 Iterative ECO Verification and Signoff

After applying ECO changes to the design in ICC2, it is critical to recheck the design in PrimeTime to ensure that all timing and DRC violations have been resolved. This iterative process continues until the design meets all required constraints and is ready for signoff

10 Implemented PULPino SoC

10.1. Choosing RISCY core

To achieve higher speed DSP grade MCU the RISCY variation of PULPino were selected.

RISCY is one of the cores developed under the PULP project. It is a high-performance 32-bit RISC-V core designed for efficiency and speed, also It includes custom DSP extensions to accelerate digital signal processing tasks.

Also, it is a 4-stage pipeline core designed to balance performance and power consumption, making it suitable for computationally intensive tasks while maintaining energy efficiency.

PULPino integrates the RISCY core as its processing unit, benefiting from its performance and low-power features. This combination is particularly effective for DSP processors thus making the high-speed target feasible, that need to process data efficiently while conserving power.[2]

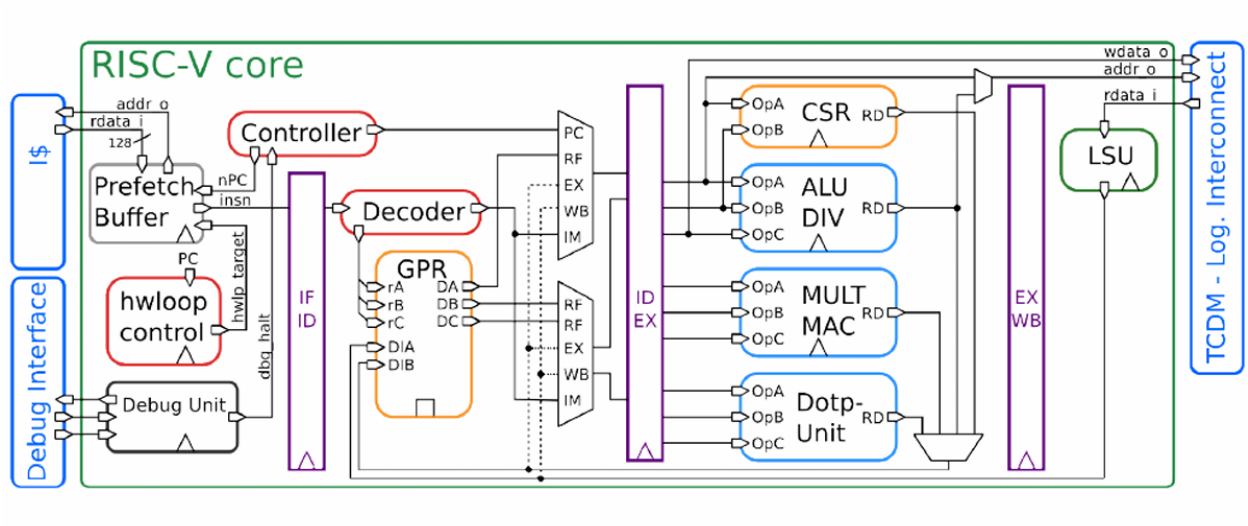

Figure 17 RISCY core

10.2. Design Constraints (SDC file)

The following constraints were used to constrain the final implementation of the PULPino SoC, the variation.

Main clock (Core & Memory)	1.5 ns (666.67 MHz)
Debugging clock	4 ns (250 MHz)
SPI clock	4 ns (250 MHz)
Input delay	0.1 ns
Output delay	0.1 ns
Input driving cell	SAEDRVT14_BUF_4
Max corner	ss0p6vm40c
Min corner	ff0p88v25c

10.3. Synthesis

To achieve an optimized design for the PULPino SoC, this thesis employs both flat and hierarchical synthesis flows within Synopsys Design Compiler. The flat flow synthesizes the entire design as a single entity, aiming for straightforward timing and area optimization. Conversely, the hierarchical flow breaks down the design into smaller modules, synthesizing each separately to potentially enhance performance and manage complexity. By comparing the final results from these two approaches, including metrics such as timing, area, and power consumption, we will determine the more effective synthesis strategy for our target performance objectives.[11]

Useful commands to achieve timing optimization in synthesis

To achieve a high-performance System on Chip (SoC), a specific set of application options and commands are utilized during the synthesis.

set_cost_priority -delay

Purpose: This command sets the optimization priority to focus on improving timing delays within the design. By prioritizing delay in the optimization process, Design Compiler directs its efforts towards reducing critical path delays. The critical path is the longest path of sequential logic between two points in the design, and reducing its delay is crucial for meeting timing constraints and achieving high performance.

compile_ultra

Purpose: initiates an aggressive synthesis and optimization process aimed at achieving optimal design performance.triggers a comprehensive set of synthesis and optimization techniques that go beyond standard optimizations. It applies advanced algorithms and heuristics to reduce timing violations and improve overall design performance.

compile_ultra -incremental

Purpose: performs further optimizations in an incremental manner, leveraging previous synthesis results.preserving previously achieved optimizations while refining timing closure further.

Flat Flow results

Clocks Report

```
****************************************
Report : clocks
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Mon Jun 17 17:56:58 2024
****************************************

Attributes:
    d - dont_touch_network
    f - fix_hold
    p - propagated_clock
    G - generated_clock
    g - lib_generated_clock

Clock          Period   Waveform            Attrs     Sources
-------------------------------------------------------------------------------
clk              1.50   {0 0.75}            d         {clk}
inverted_adbg_CLK
                 4.00   {2 4}               G d       {core_region_i_adv_dbg_if_i_cluster_tap_i_u_clk_inv/clk_o}
inverted_spi_slave_CLK
                 4.00   {2 4}               G d       {peripherals_i_axi_spi_slave_i_axi_spi_slave_i_u_txreg_clk_inv_i/clk_o}
spi_clk_i        4.00   {0 2}               d         {spi_clk_i}
tck_i            4.00   {0 2}               d         {tck_i}
-------------------------------------------------------------------------------

Generated     Master         Generated      Master         Waveform
Clock         Source         Source         Clock          Modification
-------------------------------------------------------------------------------
inverted_adbg_CLK
              tck_i          {core_region_i_adv_dbg_if_i_cluster_tap_i_u_clk_inv/clk_o}
                                            tck_i          divide_by(1)
inverted_spi_slave_CLK
              spi_clk_i      {peripherals_i_axi_spi_slave_i_axi_spi_slave_i_u_txreg_clk_inv_i/clk_o}
                                            spi_clk_i      divide_by(1)
-------------------------------------------------------------------------------
1
```

Figure 18 clock report

Area Report

```
****************************************
Report : area
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Mon Jun 17 17:56:43 2024
****************************************

Library(s) Used:

    saed14rvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_RVT_v_062020/stdcell_rvt/db_nldm/saed14rvt_ss0p6vm40c.db)
    saed14slvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_SLVT_v_062020/stdcell_slvt/db_nldm/saed14slvt_ss0p6vm40c.db)
    saed14lvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_LVT_v_062020/stdcell_lvt/db_nldm/saed14lvt_ss0p6vm40c.db)
    saed14hvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_HVT_v_062020/stdcell_hvt/db_nldm/saed14hvt_ss0p6vm40c.db)

Number of ports:                          1376
Number of nets:                          66657
Number of cells:                         64675
Number of combinational cells:           55189
Number of sequential cells:               9481
Number of macros/black boxes:                0
Number of buf/inv:                       10556
Number of references:                      173

Combinational area:               27537.945601
Buf/Inv area:                      2466.508811
Noncombinational area:            12188.777181
Macro/Black Box area:                 0.000000
Net Interconnect area:            59691.592068

Total cell area:                  39726.722782
Total area:                       99418.314850
```

Figure 19 Area report

Setup report

```
Startpoint: core_region_i_RISCV_CORE_id_stage_i_prepost_useincr_ex_o_reg
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: core_region_i_axi_slice_core2axi_WITH_SLICE_axi_slice_i_aw_buffer_i_buffer_i_cg_cell_clk_en_reg
          (positive level-sensitive latch clocked by clk')
Path Group: clk
Path Type: max

Des/Clust/Port     Wire Load Model       Library
------------------------------------------------
pulpino_top        35000                 saed14rvt_ss0p6vm40c

Point                                                    Incr       Path
-----------------------------------------------------------------------------
clock clk (rise edge)                                    0.00       0.00
clock network delay (ideal)                              0.00       0.00
core_region_i_RISCV_CORE_id_stage_i_prepost_useincr_ex_o_reg/CK (SAEDSLVT14_FDPSQB_2)
                                                         0.00 #     0.00 r
core_region_i_RISCV_CORE_id_stage_i_prepost_useincr_ex_o_reg/QN (SAEDSLVT14_FDPSQB_2)
                                                         0.03       0.03 r
U23308/X (SAEDSLVT14_INV_S_2)                            0.04       0.07 f
U37774/X (SAEDLVT14_NR2_4)                               0.05       0.12 r
U23175/X (SAEDHVT14_NR2_4)                               0.04       0.16 f
U38088/X (SAEDRVT14_NR2_4)                               0.04       0.21 r
U26613/X (SAEDRVT14_AOI21_V1_4)                          0.04       0.25 f
U38819/X (SAEDLVT14_OAI21_4)                             0.04       0.28 r
U30583/X (SAEDSLVT14_AOI21_V1_4)                         0.04       0.32 f
U30415/X (SAEDRVT14_OAI21_4)                             0.04       0.36 r
U38743/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.40 r
U38607/X (SAEDSLVT14_AOI21_V1_4)                         0.04       0.43 f
U38801/X (SAEDLVT14_OAI21_4)                             0.03       0.47 r
U38266/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.49 r
U38026/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.52 r
U38090/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.54 r
U38264/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.57 r
U24575/CO (SAEDLVT14_ADDF_V2_2)                          0.03       0.60 r
U38265/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.62 r
U38089/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.65 r
U38653/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.67 r
```

```
U38652/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.70 r
U38666/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.72 r
U38665/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.75 r
U38700/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.77 r
U38699/CO (SAEDSLVT14_ADDF_V2_2)                         0.03       0.80 r
U28291/X (SAEDSLVT14_EO2_2)                              0.03       0.83 r
U28216/X (SAEDSLVT14_NR2_4)                              0.03       0.85 f
U37776/X (SAEDSLVT14_OR2_2)                              0.03       0.88 f
U38236/X (SAEDSLVT14_NR2_4)                              0.03       0.91 r
U25706/X (SAEDSLVT14_NR2_4)                              0.03       0.94 f
U45225/X (SAEDSLVT14_OR2_2)                              0.02       0.97 f
U23372/X (SAEDRVT14_ND2_MM_2)                            0.03       0.99 r
core_region_i_axi_slice_core2axi_WITH_SLICE_axi_slice_i_aw_buffer_i_buffer_i_cg_cell_clk_en_reg/D (SAEDRVT14_LDPQ_V1_2)
                                                         0.01       1.00 r
data arrival time                                                   1.00

clock clk' (rise edge)                                   0.75       0.75
clock network delay (ideal)                              0.00       0.75
clock uncertainty                                       -0.03       0.72
core_region_i_axi_slice_core2axi_WITH_SLICE_axi_slice_i_aw_buffer_i_buffer_i_cg_cell_clk_en_reg/G (SAEDRVT14_LDPQ_V1_2)
                                                         0.00       0.72 r
time borrowed from endpoint                              0.28       1.00
data required time                                                  1.00
-----------------------------------------------------------------------------
data required time                                                  1.00
data arrival time                                                  -1.00
-----------------------------------------------------------------------------
slack (MET)                                                         0.00
```

Figure 20 DC setup timing report of critical path

Hierachical Flow

Area Report

```
****************************************
Report : area
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Fri May 17 02:07:30 2024
****************************************

Library(s) Used:

    saed14hvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_HVT_v_062020/stdcell_hvt/db_nldm/saed14hvt_ss0p6vm40c.db)
    saed14slvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_SLVT_v_062020/stdcell_slvt/db_nldm/saed14slvt_ss0p6vm40c.db)
    saed14rvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_RVT_v_062020/stdcell_rvt/db_nldm/saed14rvt_ss0p6vm40c.db)
    saed14lvt_ss0p6vm40c (File: /mnt/hgfs/SAED14nm_EDK_CORE_LVT_v_062020/stdcell_lvt/db_nldm/saed14lvt_ss0p6vm40c.db)

Number of ports:                        48970
Number of nets:                         94113
Number of cells:                        64645
Number of combinational cells:          54787
Number of sequential cells:              9532
Number of macros/black boxes:               0
Number of buf/inv:                      11270
Number of references:                       8

Combinational area:              26902.581507
Buf/Inv area:                     2636.383208
Noncombinational area:           12253.423583
Macro/Black Box area:                0.000000
Net Interconnect area:           60544.035752

Total cell area:                 39156.005090
Total area:                      99700.040842
```

Figure 21 Hierarchical flow area report

Setup Report

```
Startpoint: core_region_i/RISCV_CORE/id_stage_i/alu_operand_b_ex_o_reg_5_
            (rising edge-triggered flip-flop clocked by clk)
Endpoint: core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/cg_cell/clk_en_reg
          (negative level-sensitive latch clocked by clk)
Path Group: clk
Path Type: max

Des/Clust/Port     Wire Load Model       Library
------------------------------------------------
pulpino_top        35000                 saed14rvt_ss0p6vm40c

Point                                                    Incr       Path
-----------------------------------------------------------------------
clock clk (rise edge)                                    0.00       0.00
clock network delay (ideal)                              0.00       0.00
core_region_i/RISCV_CORE/id_stage_i/alu_operand_b_ex_o_reg_5_/CK (SAEDRVT14_FDPSQB_2)
                                                         0.00 #     0.00 r
core_region_i/RISCV_CORE/id_stage_i/alu_operand_b_ex_o_reg_5_/QN (SAEDRVT14_FDPSQB_2)
                                                         0.06       0.06 r
core_region_i/RISCV_CORE/id_stage_i/alu_operand_b_ex_o[5] (riscv_id_stage_N_HWLP2_PULP_SECURE0_FPU0_APU0_SHARED_FP0_SHARED_DSP_MULT0_SHARED_INT_DIV0_SHARED_FP_DIVSQRT2_WAPUTYPE0
_APU_NARGS_CPU3_APU_WOP_CPU6_APU_NDSFLAGS_CPU15_APU_NUSFLAGS_CPU5)
                                                         0.00       0.06 r
core_region_i/RISCV_CORE/load_store_unit_i/operand_b_ex_i[5] (riscv_load_store_unit)
                                                         0.00       0.06 r
core_region_i/RISCV_CORE/load_store_unit_i/U65/X (SAEDRVT14_INV_S_2)
                                                         0.04       0.10 f
core_region_i/RISCV_CORE/load_store_unit_i/U237/X (SAEDLVT14_NR2_4)
                                                         0.03       0.13 r
core_region_i/RISCV_CORE/load_store_unit_i/U88/X (SAEDRVT14_NR2_4)
                                                         0.03       0.17 f
core_region_i/RISCV_CORE/load_store_unit_i/U151/X (SAEDSLVT14_OAI21_4)
                                                         0.04       0.21 r
core_region_i/RISCV_CORE/load_store_unit_i/U215/X (SAEDSLVT14_AOI21_V1_4)
                                                         0.04       0.24 f
core_region_i/RISCV_CORE/load_store_unit_i/U242/X (SAEDLVT14_OAI21_4)
                                                         0.02       0.27 r
core_region_i/RISCV_CORE/load_store_unit_i/U216/X (SAEDSLVT14_AOI21_V1_4)
                                                         0.04       0.30 f
core_region_i/RISCV_CORE/load_store_unit_i/U130/X (SAEDSLVT14_OA211_2)
                                                         0.03       0.34 f
core_region_i/RISCV_CORE/load_store_unit_i/U128/X (SAEDSLVT14_OA21_MM_2)
```

```
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/axi_slave_w_valid_i (axi_slice_AXI_ADDR_WIDTH32_AXI_DATA_WIDTH32_AXI_USER_WIDTH1_AXI_ID_WIDTH2_SLICE_DEPTH2)
                                                         0.00       0.95 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/slave_valid_i (axi_w_buffer_DATA_WIDTH32_USER_WIDTH1_BUFFER_DEPTH2_1)
                                                         0.00       0.95 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/valid_i (generic_fifo_DATA_WIDTH38_DATA_DEPTH2_2)
                                                         0.00       0.95 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/U3/X (SAEDSLVT14_AN2_MM_2)
                                                         0.02       0.97 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/cg_cell/en_i (cluster_clock_gating_23)
                                                         0.00       0.97 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/cg_cell/U3/X (SAEDLVT14_OR2_2)
                                                         0.03       1.00 f
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/cg_cell/clk_en_reg/D (SAEDRVT14_LDNQ_V1_4)
                                                         0.01       1.00 f
data arrival time                                                   1.00

clock clk (fall edge)                                    0.75       0.75
clock network delay (ideal)                              0.00       0.75
clock uncertainty                                       -0.03       0.72
core_region_i/axi_slice_core2axi/WITH_SLICE_axi_slice_i/w_buffer_i/buffer_i/cg_cell/clk_en_reg/G (SAEDRVT14_LDNQ_V1_4)
                                                         0.00       0.72 f
time borrowed from endpoint                              0.28       1.00
data required time                                                  1.00
-------------------------------------------------------------------------
data required time                                                  1.00
data arrival time                                                  -1.00
-------------------------------------------------------------------------
slack (MET)                                                         0.00
```

Figure 22 DC Hierarchical flow setup report

10.4. PLACE & ROUTE

Floorplan

initialize_floorplan -core_utilization 0.7 -flip_first_row true -core_offset {13.5 13.5 13.5 13.5 }

We have initialized the floorplan with 0.7 utilization.

Useful commands to achieve timing optimization

create_placement -floorplan -timing_driven

By specifying `-timing_driven`, the placement process takes into account timing considerations as a primary optimization goal. This means that during placement:

The tool prioritizes minimizing critical path delays and meeting timing constraints.

It strategically places cells to reduce wire delays and optimize signal paths between sequential elements (like flip-flops and registers).

Placement Constraints: Timing-driven placement considers constraints such as:

- **Critical Paths:** Placing critical path elements close together to minimize delay.
- **Clock Paths:** Ensuring clock signals propagate efficiently without skew or excessive delay.
- **Signal Paths:** Optimizing routing to minimize signal transition times and ensure setup and hold time requirements are met.

Post-Floorplanning:

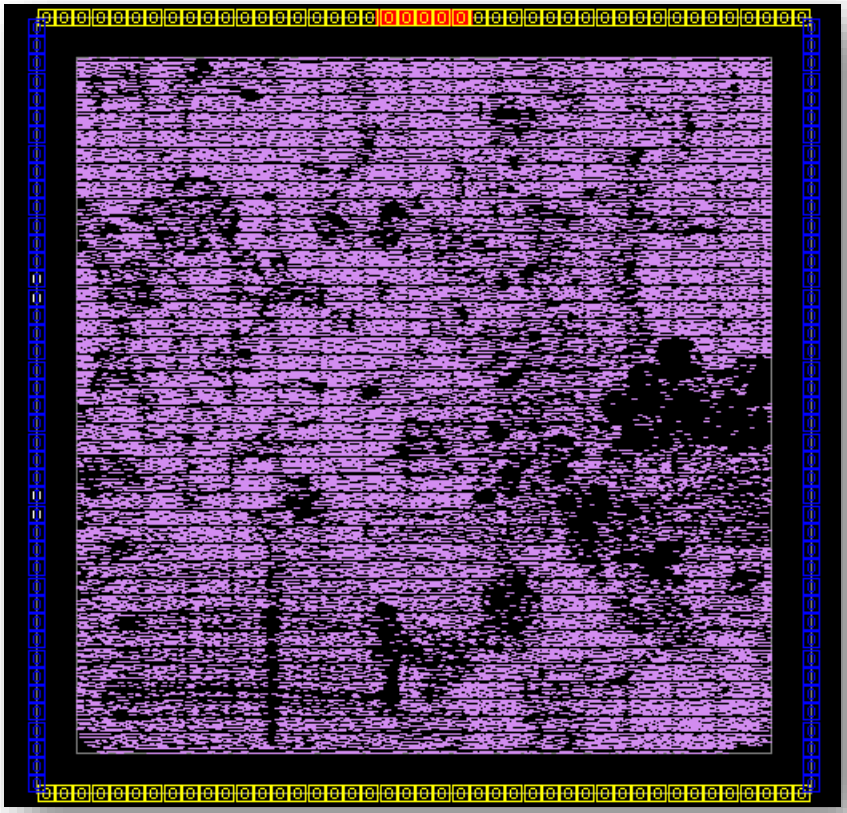

Figure 23 post floor planning and early placement

Power Plan

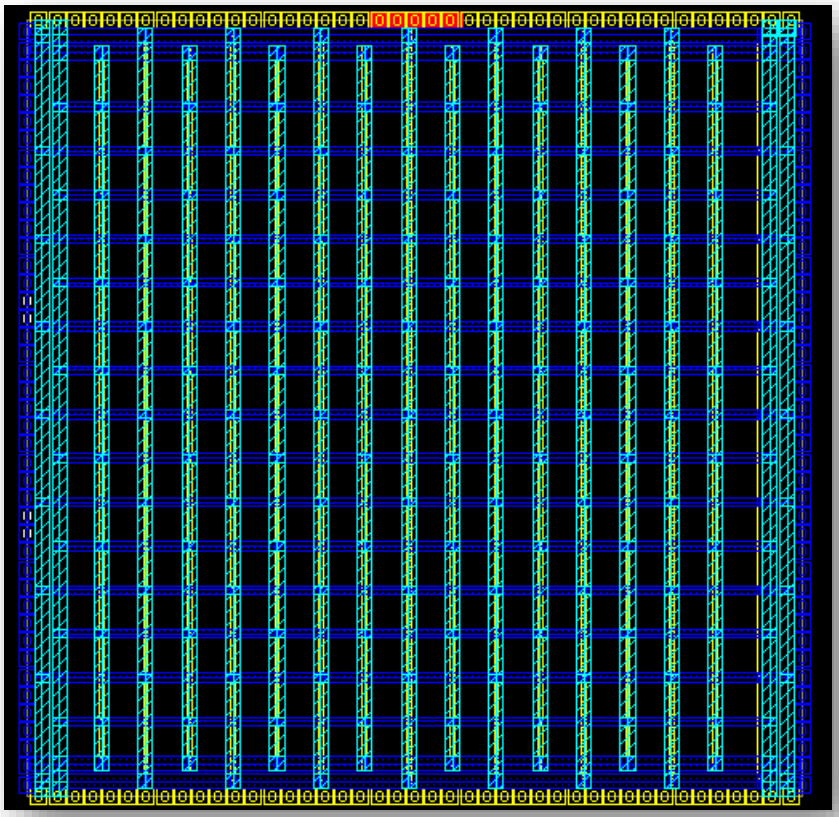

Figure 24 power mesh of the PULPIno SoC

Placement

Useful App Options to achieve timing optimization

place_opt.flow.optimize_icgs:

Enables optimization of Incremental Clock Grouping Strategy (ICGS) during placement.

Explanation: ICGS optimizes the placement of flip-flops belonging to the same clock group in close proximity. This reduces clock skew and improves setup and hold times for timing paths associated with these clock domains. By optimizing ICGS, you enhance the timing performance of synchronous designs.

place_opt.flow.optimize_ndr:

Enables optimization of Non-default Routing (NDR) during placement.

Explanation: NDR optimization focuses on improving routing congestion and delay, which are critical for meeting timing constraints. It helps in reducing routing delay and improves signal arrival times across critical paths.

place_opt.flow.do_path_opt:

Enables path-based optimization during placement.

Explanation: Path-based optimization considers critical timing paths identified during timing analysis and optimizes placement to minimize delay along these paths. It helps in achieving better timing closure by focusing on critical paths that require specific attention to meet timing requirements.

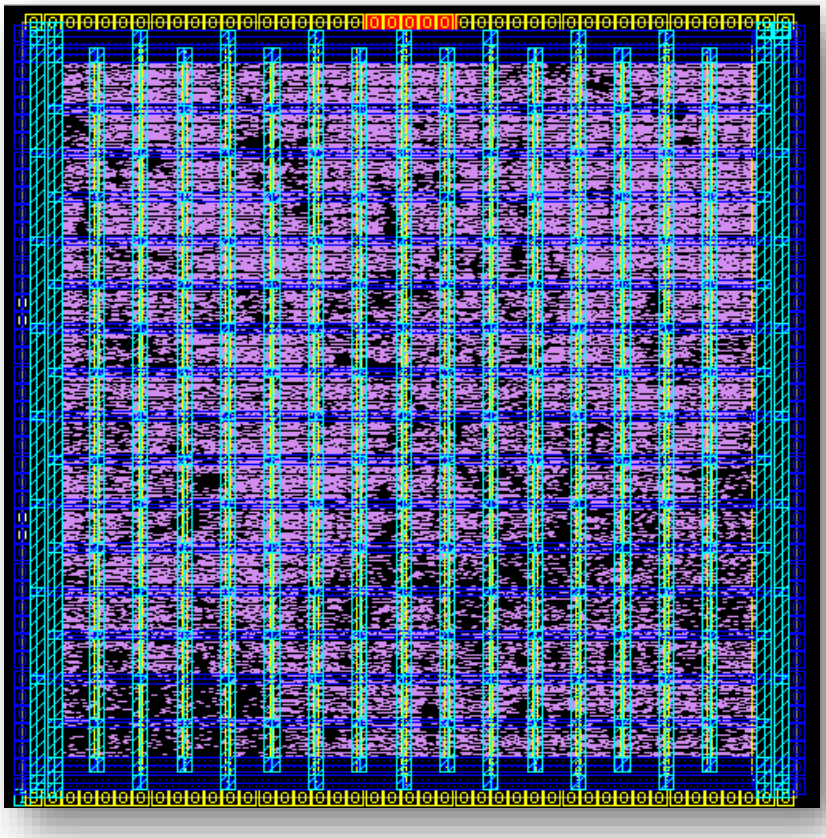

Figure 25 PULPINO SoC after timing driven placement

CTS

Useful App Options to achieve timing optimization

set_app_options -name cts.compile.enable_global_route -value true

Enables global routing during clock tree synthesis (CTS). Global routing plays a crucial role in ensuring that clock signals are routed efficiently and meet timing requirements across the entire chip. By enabling global routing, the tool can optimize clock paths more effectively, thereby improving timing closure.

set_app_options -name clock_opt.flow.optimize_ndr -value true

This option enables optimization for negative domain crossing (NDR) during clock tree optimization. NDR optimization is essential for ensuring that clock signals cross between different clock domains without violating timing constraints. Optimizing NDR helps in reducing clock skew and improving overall timing performance.

set_app_options -name ccd.skip_path_groups -value {reg2mem mem2reg}

This option specifies which path groups to skip during clock concurrent delay (CCD) analysis. CCD analysis helps in identifying and resolving timing violations caused by clock interactions. By skipping certain path groups (`reg2mem` and `mem2reg`), the tool focuses on optimizing other critical timing paths, thereby improving timing closure efficiency.

set_app_options -name clock_opt.enable_ccd -value true

This option enables clock concurrent delay (CCD) analysis during optimization. CCD analysis helps in identifying and resolving timing violations caused by clock interactions. By enabling CCD, the tool can optimize timing paths affected by clock skew, jitter, or other timing-related issues related to clock distribution and synchronization.

set_app_options -name ccd.hold_control_effort -value high

This option controls the effort level for hold timing optimization during CCD analysis. Hold timing optimization ensures that data signals arriving at flip-

flops meet setup and hold timing requirements. Setting the effort level to `high` directs the tool to put more emphasis on optimizing hold timing.

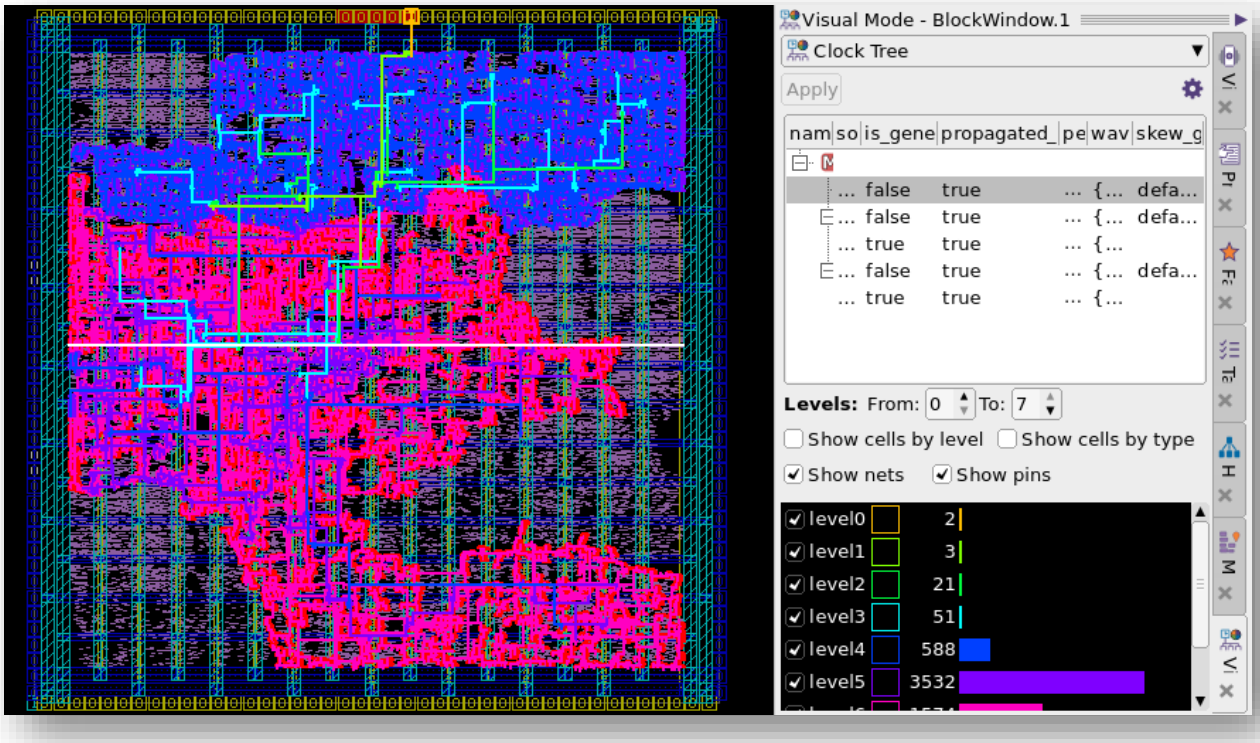

Figure 26 PULPINO SoC Clock tree

Reports

Utilization Report

```
****************************************
Report : report_utilization
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Wed Jun 19 15:24:49 2024
****************************************
Utilization Ratio:                      0.7380
Utilization options:
 - Area calculation based on:           site_row of block pulpino_top_5_cts
 - Categories of objects excluded:      hard_macros macro_keepouts soft_macros io_cells hard_blockages
Total Area:                             56740.6692
Total Capacity Area:                    56740.6692
Total Area of cells:                    41873.1960
Area of excluded objects:
 - hard_macros         :                0.0000
 - macro_keepouts      :                0.0000
 - soft_macros         :                0.0000
 - io_cells            :                0.0000
 - hard_blockages      :                0.0000

Utilization of site-rows with:
 - Site 'unit':                         0.7380

0.7380
```

Figure 27 PULPINO SoC post CTS utilization

Clock Area Report

```
==========================
==== Area (per clock) ====
==========================

Clock                                 Sinks          Clock          Clock          Clock          Clock          Sinks           Sink
                                  (balanced)     Cell Count        Stdcell       Repeater       Repeater     (physical)        Stdcell
Macro
                                                  (non sink)           Area          Count           Area                        Area
Area
-------------------------------------------------------------------------------------------------------------------------------------------
------------
clk                                     8572            535         153.67            499         130.00           8574       11023.45
0.00
spi_clk_i                                496             33           9.90             30           8.61            496         638.65
0.00
 inverted_spi_slave_CLK                   44              6           2.44              5           1.82             44          56.65
0.00
tck_i                                    409             29           7.95             25           6.44            409         526.63
0.00
 inverted_adbg_CLK                         5              2           0.93              0           0.00              5           6.44
0.00
-------------------------------------------------------------------------------------------------------------------------------------------
------------
Total                                   9477            597         171.52            554         145.05           9479       12188.73
0.00
```

Figure 28 PULPINO SoC clock area report

Timing Reports

```
Scenario            'func_fast'
Timing Path Group   'clk'
----------------------------------------
Levels of Logic:                      38
Critical Path Length:               0.29
Critical Path Slack:                0.41
Critical Path Clk Period:           1.50
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:              -0.00
Total Hold Violation:              -0.01
No. of Hold Violations:              144
----------------------------------------

Scenario            'func_fast'
Timing Path Group   'spi_clk_i'
----------------------------------------
Levels of Logic:                       6
Critical Path Length:               0.07
Critical Path Slack:                1.89
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:              -0.00
Total Hold Violation:              -0.00
No. of Hold Violations:               12
----------------------------------------

Scenario            'func_fast'
Timing Path Group   'tck_i'
----------------------------------------
Levels of Logic:                       6
Critical Path Length:               0.15
Critical Path Slack:                1.80
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:              -0.00
Total Hold Violation:              -0.00
No. of Hold Violations:                8
----------------------------------------
```

```
Scenario            'func_slow'
Timing Path Group   'clk'
----------------------------------------
Levels of Logic:                      36
Critical Path Length:               0.68
Critical Path Slack:               -0.00
Critical Path Clk Period:           1.50
Total Negative Slack:              -0.00
No. of Violating Paths:                7
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'spi_clk_i'
----------------------------------------
Levels of Logic:                       6
Critical Path Length:               0.30
Critical Path Slack:                1.63
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'tck_i'
----------------------------------------
Levels of Logic:                       5
Critical Path Length:               0.53
Critical Path Slack:                1.39
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
```

Figure 29 timing report post CTS very small hold violation to be fixed with primetime ECO flow

Routing

Useful App Options to achieve timing optimization

route.global.timing_driven:

this option indicates that global routing should be timing-driven. Global routing sets up the initial routing paths and is also influenced by timing constraints to ensure that subsequent detailed routing has a good starting point.

route.track.timing_driven:

This option specifies that track routing (often used in advanced nodes where routing resources are constrained) should be timing-driven. Track routing

optimizes the routing paths within predefined tracks to meet timing constraints efficiently.

route.detail.timing_driven:

Similar to `route.global.timing_driven`, This option indicates that detailed routing should be timing-driven. Timing-driven routing ensures that the routing paths are optimized to meet timing constraints, such as setup and hold times. It's crucial for achieving timing closure in the physical implementation stage.

time.si_enable_analysis:

This option enables signal integrity (SI) analysis, which is critical for ensuring that signals propagate correctly without timing violations due to issues like crosstalk or signal reflection. SI analysis helps in identifying and mitigating timing issues caused by signal integrity problems.

route.detail.drc_convergence_effort_level -value high:

This option sets the convergence effort level for DRC (Design Rule Checking) during detailed routing to high. DRC checks ensure that the layout adheres to the manufacturing rules and specifications. While not directly optimizing timing, setting a high convergence effort level can indirectly impact timing by ensuring that routing meets critical DRC requirements efficiently, thereby potentially reducing routing iterations that could affect timing closure.

route.common.rc_driven_setup_effort_level -value high:

This option sets the setup effort level for RC (Resistance-Capacitance) driven routing to high. RC-driven routing considers the parasitic resistances and capacitances in the routing paths, which directly affect timing by influencing signal propagation delays. Higher setup effort levels in RC-driven routing typically aim to optimize routing paths to minimize these delays and improve timing performance.

Reports

LVS Report

```
==============================================================
    Maximum number of violations is set to 5000
    Abort checking when more than 5000 violations are found
    All violations might not be found.
==============================================================
Total number of input nets is 69725.
Total number of short violations is 0.
Total number of open nets is 0.
Total number of floating route violations is 0.

Elapsed =    0:00:35, CPU =    0:00:35
1
```

Figure 30 PULPINO SoC LVS clean

Check Routes

```
Verify Summary:

Total number of nets = 69725, of which 0 are not extracted
Total number of open nets = 0, of which 0 are frozen
Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0 nets
                                 0 ports without pins of 0 cells connected to 0 nets
                                 0 ports of 0 cover cells connected to 0 non-pg nets
Total number of DRCs = 0
Total number of antenna violations = no antenna rules defined
Total number of tie to rail violations = not checked
Total number of tie to rail directly violations = not checked
```

Figure 31 zroute DRCs report

Finishing

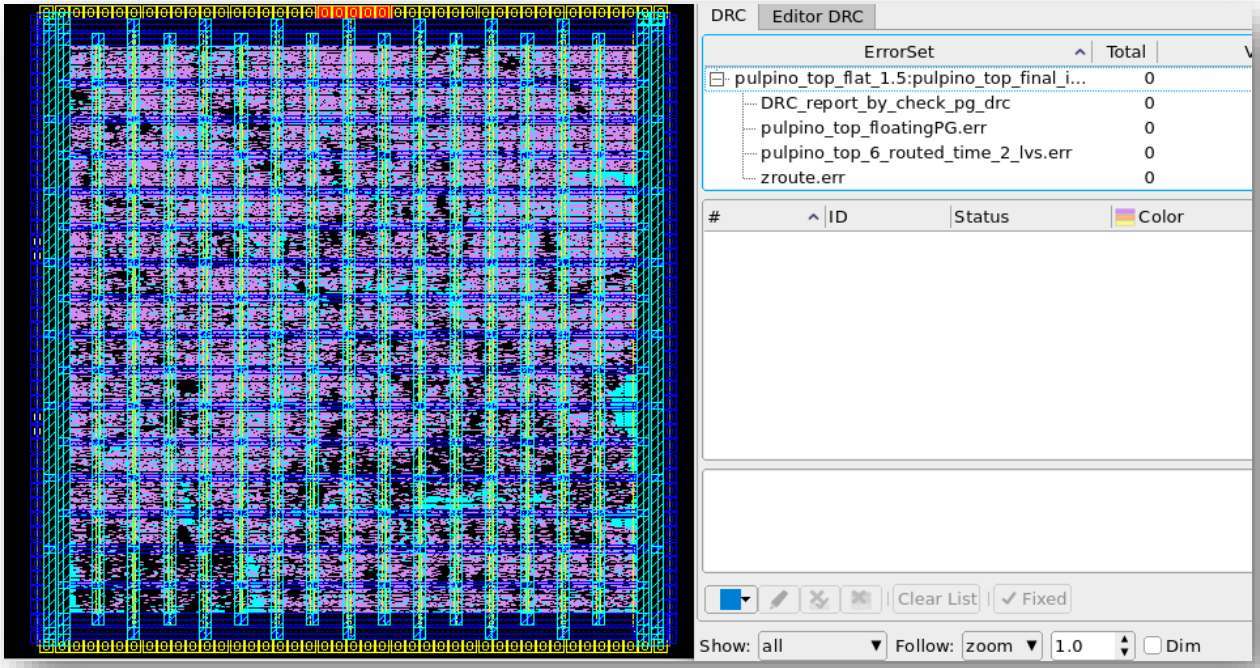

Figure 32 finished chip no physical DRCs

Chip finishing stage included inserting filler cells & DCAP cells and resolving any legality issues.

Check Legality

```
check_legality for block design pulpino_top succeeded!


check_legality succeeded.

************************
```

Figure 33 legalized design after chip finishing

10.5. Parasitic Extraction using StarRC

To ensure accurate static timing analysis and signal integrity verification, we employed Synopsys StarRC for parasitic extraction in our design flow. StarRC is a comprehensive tool that calculates the parasitic resistances and capacitances in an integrated circuit layout. These parasitics are crucial for accurate timing, power, and noise analysis, as they impact signal propagation delays and overall circuit performance.

Role of Parasitic Extraction

Parasitic extraction involves determining the unintended resistive and capacitive effects within the interconnects and devices of a semiconductor design. Accurate extraction of these parasitics is essential for:

StarRC and SPEF Generation

StarRC performs detailed parasitic extraction and generates a Standard Parasitic Exchange Format (SPEF) file, which encapsulates the extracted Parasitics in a standardized format. The SPEF file serves as a critical input for the timing signoff tool Primetime, enabling them to incorporate real-world parasitic effects into their analysis.

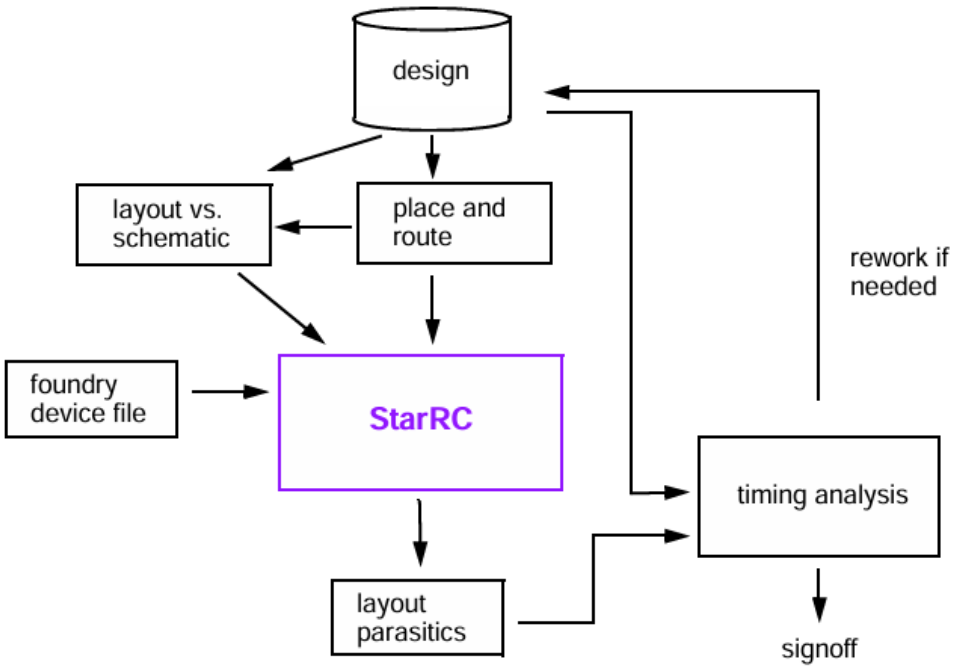

Figure 34 StarRC flow

10.6. Fixing timing and logical DRCs using Prime time ECO flow

Pre ECO

After finishing routing stage multiple hold time , setup time , max capacitance violations have been encountered

```
Constraint                                          Cost
-----------------------------------------------------
max_delay/setup                                    0.002  (VIOLATED)
min_delay/hold                                     1.625  (VIOLATED)
recovery                                           0.000  (MET)
removal                                            0.487  (VIOLATED)
clock_gating_setup                                 0.000  (MET)
clock_gating_hold                                  0.000  (MET)
sequential_clock_pulse_width                       0.000  (MET)
clock_tree_pulse_width                             0.000  (MET)
max_capacitance                                   37.577  (VIOLATED)
min_capacitance                                    8.250  (VIOLATED)
max_fanout                                         0.000  (MET)
```

Figure 35:report constraints pre-ECO

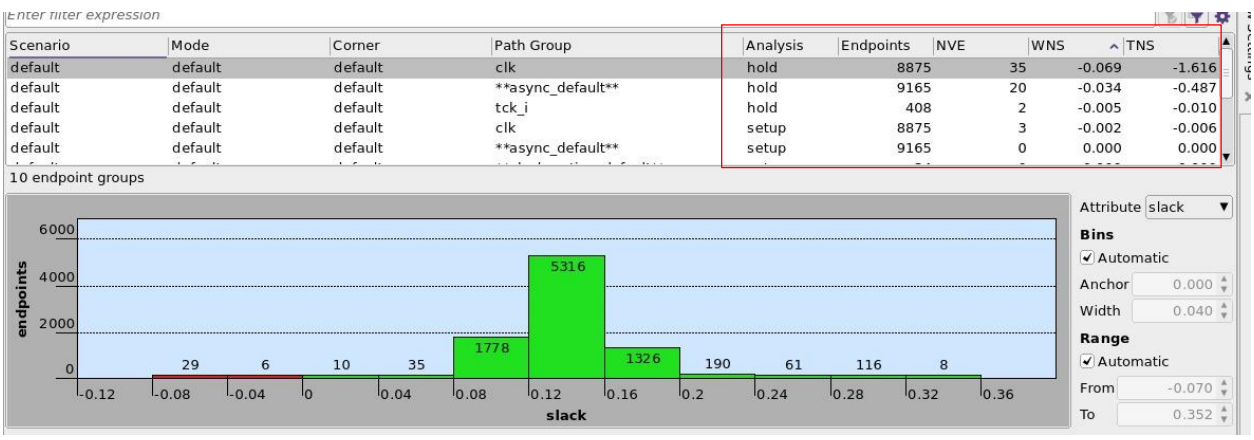

Figure 36: paths slack pre Timing fixing

Post ECO

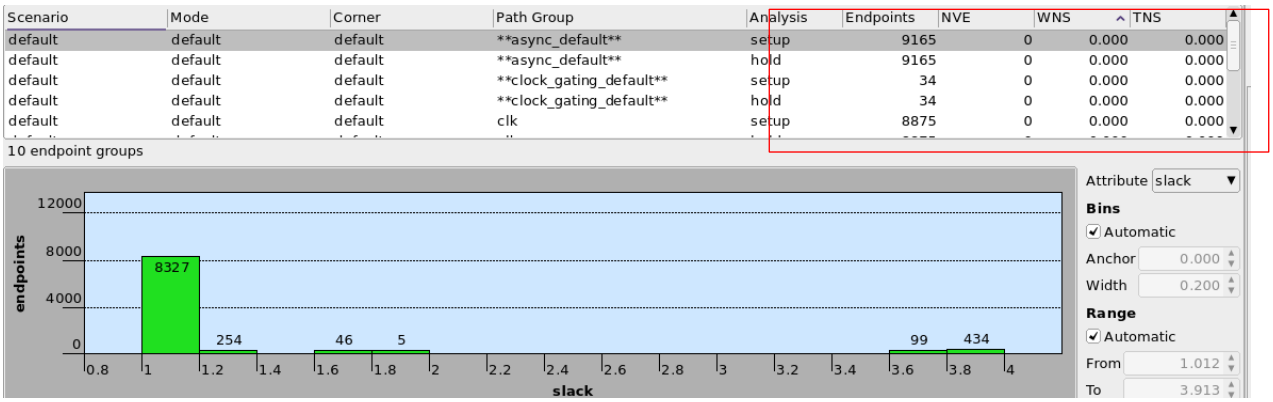

Figure 37 Paths slack after ECO fixing

```
Constraint                                          Cost
-----------------------------------------------------------
max_delay/setup                                    0.000  (MET)
min_delay/hold                                     0.000  (MET)
recovery                                           0.000  (MET)
removal                                            0.000  (MET)
clock_gating_setup                                 0.000  (MET)
clock_gating_hold                                  0.000  (MET)
sequential_clock_pulse_width                       0.000  (MET)
clock_tree_pulse_width                             0.000  (MET)
max_capacitance                                    0.000  (MET)
min_capacitance                                    8.250  (VIOLATED)
max_fanout                                         0.000  (MET)
```

Figure 38 report constraints after ECO fixing

10.7. Flat vs Hierarchical flows Quality of Results

Fast timing corner

```
Report : qor
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Mon Jun 24 14:17:25 2024
****************************************


Scenario           'func_fast'
Timing Path Group  'clk'
----------------------------------------
Levels of Logic:                     70
Critical Path Length:              0.57
Critical Path Slack:               0.90
Critical Path Clk Period:          1.50
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

```
Scenario           'func_fast'
Timing Path Group  'spi_clk_i'
----------------------------------------
Levels of Logic:                      7
Critical Path Length:              0.10
Critical Path Slack:               1.89
Critical Path Clk Period:          4.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

```
Scenario           'func_fast'
Timing Path Group  'inverted_adbg_CLK'
----------------------------------------
Levels of Logic:                     21
Critical Path Length:              0.29
Critical Path Slack:               1.67
Critical Path Clk Period:          4.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

```
Scenario           'func_fast'
Timing Path Group  'tck_i'
----------------------------------------
Levels of Logic:                      5
Critical Path Length:              0.15
Critical Path Slack:               1.84
Critical Path Clk Period:          4.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

```
Scenario           'func_fast'
Timing Path Group  'tck_i'
----------------------------------------
Levels of Logic:                      5
Critical Path Length:              0.15
Critical Path Slack:               1.84
Critical Path Clk Period:          4.00
Total Negative Slack:              0.00
No. of Violating Paths:               0
Worst Hold Violation:              0.00
Total Hold Violation:              0.00
No. of Hold Violations:               0
----------------------------------------
```

Figure 39 Flat Flow Fast time corner

Slow timing corner

```
Scenario            'func_slow'
Timing Path Group   'clk'
----------------------------------------
Levels of Logic:                      60
Critical Path Length:               1.48
Critical Path Slack:                0.00
Critical Path Clk Period:           1.50
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'spi_clk_i'
----------------------------------------
Levels of Logic:                       4
Critical Path Length:               0.39
Critical Path Slack:                1.57
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'tck_i'
----------------------------------------
Levels of Logic:                       5
Critical Path Length:               0.56
Critical Path Slack:                1.38
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------
```

```
Scenario            'func_slow'
Timing Path Group   'tck_i'
----------------------------------------
Levels of Logic:                       5
Critical Path Length:               0.56
Critical Path Slack:                1.38
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'inverted_spi_slave_CLK'
----------------------------------------
Levels of Logic:                       4
Critical Path Length:               0.46
Critical Path Slack:                1.54
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------

Scenario            'func_slow'
Timing Path Group   'inverted_adbg_CLK'
----------------------------------------
Levels of Logic:                      21
Critical Path Length:               1.03
Critical Path Slack:                0.95
Critical Path Clk Period:           4.00
Total Negative Slack:               0.00
No. of Violating Paths:                0
Worst Hold Violation:               0.00
Total Hold Violation:               0.00
No. of Hold Violations:                0
----------------------------------------
```

Figure 40 Flat Flow slow corner

Flat flow Area

```
Area
----------------------------------------
Combinational Area:              29731.79
Noncombinational Area:           12188.73
Buf/Inv Area:                     4451.19
Total Buffer Area:                2106.20
Total Inverter Area:              2344.99
Macro/Black Box Area:                0.00
Net Area:                               0
Net XLength:                         0.00
Net YLength:                         0.00
----------------------------------------
Cell Area (netlist):                          41920.53
Cell Area (netlist and physical only):        56740.67
Net Length:                          0.00
```

Figure 41 Flat flow area

Flat flow Power

Power Group	Internal Power	Switching Power	Leakage Power	Total Power	(%)	Attrs
io_pad	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)	
memory	0.00e+00	0.00e+00	0.00e+00	0.00e+00	(0.0%)	
black_box	0.00e+00	0.00e+00	7.01e+04	7.01e+04	(0.0%)	
clock_network	1.53e+09	7.62e+08	8.43e+03	2.29e+09	(94.1%)	
register	4.40e+07	2.76e+06	8.52e+04	4.68e+07	(1.9%)	
sequential	5.22e+05	2.16e+03	6.10e+01	5.24e+05	(0.0%)	
combinational	1.29e+07	8.42e+07	1.98e+05	9.73e+07	(4.0%)	
Total	1.59e+09 pW	8.49e+08 pW	3.61e+05 pW	2.44e+09 pW		

Figure 42 flat flow power

Violating constrains

```
icc2_shell> report_constraints -all_violators
****************************************
Report : constraint
         -all_violators
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Mon Jun 24 15:06:52 2024
****************************************

   late_timing
   -----------

Endpoint                          Path Delay     Path Required       CRP    Slack Group    Scenario
-------------------------------------------------------------------------------------------------------
No paths.

   early_timing
   -----------

Endpoint                          Path Delay     Path Required       CRP    Slack Group    Scenario
-------------------------------------------------------------------------------------------------------
No paths.

   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------------
   Number of max_transition violation(s): 0

   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------------
   Number of max_capacitance violation(s): 0


   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------------
   Number of min_capacitance violation(s): 0

   Total number of violation(s): 0
```

Figure 43 Flat flow constraints reports

Utilization

```
-----------------------------------------
Utilization Ratio:                      0.7151
Utilization options:
 - Area calculation based on:           site_row of block pulpino_pre_finish
 - Categories of objects excluded:      hard_macros macro_keepouts soft_macros io_cells hard_blockages
Total Area:                             54282.9072
Total Capacity Area:                    54282.9072
Total Area of cells:                    38816.7000
Area of excluded objects:
 - hard_macros         :                0.0000
 - macro_keepouts      :                0.0000
 - soft_macros         :                0.0000
 - io_cells            :                0.0000
 - hard_blockages      :                0.0000

Utilization of site-rows with:
 - Site 'unit':                         0.7151

0.7151
```

Figure 44 Flat flow Utilization

Hierarchical Flow timing

```
Timing Path Group 'clk' (max_delay/setup)
-------------------------------------------
Levels of Logic:                          10
Critical Path Length:                  0.979
Critical Path Slack:                   0.000
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------

Timing Path Group 'spi_clk_i' (max_delay/setup)
-------------------------------------------
Levels of Logic:                           5
Critical Path Length:                  0.544
Critical Path Slack:                   1.439
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------

Timing Path Group 'tck_i' (max_delay/setup)
-------------------------------------------
Levels of Logic:                          37
Critical Path Length:                  1.507
Critical Path Slack:                   0.473
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------
```

```
Timing Path Group 'clk' (min_delay/hold)
-------------------------------------------
Levels of Logic:                           4
Critical Path Length:                  0.180
Critical Path Slack:                   0.002
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------

Timing Path Group 'spi_clk_i' (min_delay/hold)
-------------------------------------------
Levels of Logic:                           1
Critical Path Length:                  0.099
Critical Path Slack:                   0.027
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------

Timing Path Group 'tck_i' (min_delay/hold)
-------------------------------------------
Levels of Logic:                           2
Critical Path Length:                  0.085
Critical Path Slack:                   0.008
Total Negative Slack:                  0.000
No. of Violating Paths:                    0
-------------------------------------------
```

Figure 45 Hierarchical timing

Hierchical Flow Area

```
Area
----------------------------------------
Combinational Area:            29153.79
Noncombinational Area:         12253.29
Buf/Inv Area:                   4774.69
Total Buffer Area:              2651.04
Total Inverter Area:            2123.65
Macro/Black Box Area:              0.00
Net Area:                             0
Net XLength:                       0.00
Net YLength:                       0.00
----------------------------------------
Cell Area (netlist):                          41407.08
Cell Area (netlist and physical only):        51813.47
Net Length:                        0.00
```

Figure 46 Hierchical Area

Hierchical Flow Power

```
  Cell Internal Power    = 1.58e+09 pW ( 65.4%)
  Net Switching Power    = 8.34e+08 pW ( 34.6%)
Total Dynamic Power      = 2.41e+09 pW (100.0%)

Cell Leakage Power       = 3.79e+05 pW


  Attributes
  ----------
      u  -  User defined power group

Power Group         Internal Power        Switching Power          Leakage Power            Total Power    (   %  )    Attrs
--------------------------------------------------------------------------------------------------------------------------------
io_pad                  0.00e+00              0.00e+00               0.00e+00               0.00e+00    (  0.0%)
memory                  0.00e+00              0.00e+00               0.00e+00               0.00e+00    (  0.0%)
black_box               0.00e+00              0.00e+00               7.36e+04               7.36e+04    (  0.0%)
clock_network           1.52e+09              7.53e+08               8.20e+03               2.27e+09    ( 94.2%)
register                4.33e+07              2.92e+06               8.96e+04               4.64e+07    (  1.9%)
sequential              4.49e+05              1.57e+03               1.94e+01               4.50e+05    (  0.0%)
combinational           1.57e+07              7.79e+07               2.07e+05               9.38e+07    (  3.9%)
--------------------------------------------------------------------------------------------------------------------------------
Total                   1.58e+09 pW           8.34e+08 pW            3.79e+05 pW            2.41e+09 pW
1
```

Figure 47 Hierchical Power

Violating constrains

```
****************************************
Report : constraint
        -all_violators
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Mon Jun 24 15:06:52 2024
****************************************

   late_timing
   -----------

Endpoint                        Path Delay     Path Required       CRP    Slack Group    Scenario
-------------------------------------------------------------------------------------------------------
No paths.

   early_timing
   -----------

Endpoint                        Path Delay     Path Required       CRP    Slack Group    Scenario
-------------------------------------------------------------------------------------------------------
No paths.

   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------
   Number of max_transition violation(s): 0

   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------
   Number of max_capacitance violation(s): 0


   Mode: func Corner: fast
   Scenario: func_fast
  ---------------------------------------------------------------------
   Number of min_capacitance violation(s): 0

   Total number of violation(s): 0
```

Figure 48 Hierarchical constraint report

```
Report : report_utilization
Design : pulpino_top
Version: O-2018.06-SP1
Date   : Thu Jun 20 22:44:47 2024
****************************************
Utilization Ratio:                      0.7395
Utilization options:
 - Area calculation based on:           site_row of block pulpino_top_6_routed_ecof
 - Categories of objects excluded:      hard_macros macro_keepouts soft_macros io_cells hard_blockages
Total Area:                             55909.5456
Total Capacity Area:                    55909.5456
Total Area of cells:                    41347.8552
Area of excluded objects:
 - hard_macros         :                0.0000
 - macro_keepouts      :                0.0000
 - soft_macros         :                0.0000
 - io_cells            :                0.0000
 - hard_blockages      :                0.0000

Utilization of site-rows with:
 - Site 'unit':                         0.7395

0.7395
```

Figure 49 Hierarchical Utilization

10.8. Conclusion

Comparing Flat vs Hierarchical flow, it is deduced that Hierarchical flow is slightly better than the flat in terms of power and area for the same operating frequency. The design have achieved 666.67MHz for the core clock with utlization of 74% and power of 2.4 mW. These Figures where achieved using the hierarchical flow. The implementation of the SoC incorporated the use of Synopsis design compiler (DC) for synthesis, IC Compiler II (ICC2) for physical implementation, StarRC for parasitic extraction for accurate interconnect delay calculation and Primetime for timing signoff.

10.9. References

[1]PULP Implementation (pulp-platform.org)

[2] PULPino datasheet

[3] PULPissimo datasheet

[4] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu,"VLSI Physical Design: From Graph Partitioning to Timing Closure," Cambridge University Press, 2011.

[5] P. P. Chu, "FPGA Prototyping by Verilog Examples,"

[6] M. Singh,"California State UniversityNorthridge Implementation ofComplete ASICDesign Flow, RTL to GDS-II," M.S. thesis, California State University, Northridge, Dec. 2011. [Online].

Available: http://hdl.handle.net/10211.3/141729

[7]"Formal Verification Basics," [Online]. Available: http://tech.tdzire.com/formal-verification-basics/

[8]"Verilog HDL: AGuide to Digital Design and Synthesis," Second Edition, IEEE 1364-2001 Verilog HDL standard, Prentice Hall PTR, 2003, ISBN: 0-13-044911-3 143

[9] S. Churiwala and S. Garg, "Principles of VLSI RTL Design," Springer, 2011, Chapter 6, ISBN978-1- 4419-9295-6

[10]N. H. E. Weste and D.Harris,"CMOS VLSIDesign," Third Edition, Pearson, Addison Wesley, 2004, Chapter 9, ISBN: 978-0321149015

[11] C.Nadella,"VLSI Physical Design Back-End Process,"[Online]. Available: https://www.slideshare.net/NADELLACHENCHU/vlsi-physical-design-back end-process.

[12]"Understanding the Importance of Prerequisites in the VLSI Physical Design Stage," Design & Reuse, [Online]. Available: https://www.design-reuse.com/articles/54634/understanding the-importance-of- prerequisites